Void Topos: Distinction Before Topos

Johannes Michael Wielsch
Formal development in the *Void and Form* project
Project DOI: 10.5281/zenodo.19491035

May 2026

Abstract

This paper gives a machine-checked route from binary distinction to an elementary-topos stage. The point is one of priority. In this route, the language of toposes is not used as the ambient beginning; it is recovered only after distinction-induced readings have been constructed, tested, and quotient-completed.

The development is self-contained relative to the Agda standard library: it imports only the infrastructure needed for universes, identity types, coproducts, dependent pairs, the empty type, and negation. No result from the main `Void` development is used.

The checked route has three main movements. First, every binary distinction is shown to have a normal form up to boundary-preserving isomorphism, and this normal form is lifted to distinction-induced stable readings. Second, the raw category of stable readings is built. It has identities, composition, a terminal reading, weak binary products, and a displayed classifier extension for the mono-like subcarriers defined here. It also has a formal obstruction: an explicit split-truth cospan has no pullback cone, so the full raw elementary-topos package cannot be jointly supplied. Third, a quotient/skeleton completion identifies presentation-equivalent readings in a new carrier. At that completed stage the category is terminal, and therefore carries the elementary topos structure by direct construction.

The elementary topos is therefore a certified endpoint of the route, not its starting language. The raw layer is not discarded; it is the place where categorical structure is generated and where the obstruction is proved. The completed skeleton is the first topos threshold.

The remaining parts locate what lies above that threshold. The minimal completed topos is not set-like in the displayed sense, because it has no pair of distinct objects. A richer set-like stage is treated as an additional burden: the paper constructs the canonical free syntactic completion with well-pointed structure and NNO, proves its initiality, and shows uniqueness up to equivalence among initial completions. It then makes the model notion explicit and proves that this free completion classifies the displayed models over the generated reading skeleton.

Finally, the paper records the exact strength of the foundational claim. Any admitted interface with two distinguishable formulation positions carries an embedded binary distinction. Any set-like candidate defined here carries such positions by object diversity. Route-admitted completion claims are classified as canonical skeleton, impossible raw topos attempt, set-like burden, or semantic-adequacy burden; admitted foundation claims are classified by the same route, after their admission interface has made nontrivial formulability explicit. The classification is exhaustive for these displayed interfaces; wider semantic totalities require further admission data.

How to read this paper

The paper is best read as a translation route. Standard categorical words do appear: classifier, product, pullback, exponential, topos, well-pointed, set-like, model, foundation. They do not all

enter at the same level. The route fixes their order.

First comes binary distinction: a carrier with two separated exhaustive points. Its normal form supplies the two-valued reading carrier. Next comes the raw stable-reading category. This layer already has genuine categorical structure, but it is still raw: product uniqueness is named as a boundary, global pullbacks fail, and the classifier data is only the displayed extension for the mono-like subcarriers defined here. The split-truth cospan is the decisive test of this layer.

The quotient/skeleton completion is the first topos stage. It does not pretend that old presentations were definitionally equal; it constructs a new completed carrier in which presentation equality is represented lawfully. At that endpoint the category is terminal, so the elementary topos data close by direct construction. This is the minimal topos threshold of the paper.

Above that threshold the vocabulary changes again. A set-like topos is not the minimal endpoint; it is an additional nonterminal completion burden. The free well-pointed completion with NNO is the canonical syntactic inhabitant of that burden, and its universal property gives the model classification over the generated reading skeleton. Finally, the foundation language is admitted only through a formal interface: either a claim supplies nontrivial formulation positions, or it appears as one of the route-completion claims already classified.

The results are conditional in the exact mathematical sense. Once the displayed distinction and reading data are supplied, the constructions, obstructions, completions, and classifications follow. Claims outside these admission interfaces require further data; inside the interfaces, the route assigns a formal class.

Map of the construction and classification

The map below is a dictionary for the route. It tells the reader what each layer means in the paper's vocabulary, what has been checked there, and what burden is carried forward.

Distinction. A binary distinction is a carrier with two separated points that exhaust it. Every such presentation normalizes to Two up to boundary-preserving isomorphism.

Presentation equality. The swapped two-point presentation shows that isomorphism cannot be silently read as raw fieldwise equality. A lawful completion must introduce a quotient or skeleton carrier.

Raw readings. A distinction induces a classifier-valued reading. The raw category of such readings has identities, composition, a terminal object, weak binary products, and a displayed classifier extension for the mono-like subcarriers defined here.

Obstruction. The raw category is not an elementary topos. The split-truth cospan over the two-valued classifier has no pullback cone, so global pullbacks cannot be supplied for the full raw category.

Completion topos stage. The quotient-completed skeleton stage has one object and one morphism. It therefore carries terminal object, binary products, pullbacks, exponentials, and a subobject classifier by direct construction.

Set-like burden. The minimal completed topos is not set-like, because it has no pair of distinct objects. A richer set-like completion satisfying the displayed object-diversity condition cannot remain at the terminal skeleton. The candidate package records the nonterminal burden package: point separation, arithmetic, cover sections, and semantic adequacy for the reading interface.

Free well-pointed completion. The paper constructs the canonical free syntactic inhabitant of that package: a nontrivial set-like well-pointed elementary topos with NNO over the distinction-induced reading skeleton, full and faithful reading embedding, initiality, and uniqueness up to equivalence among initial completions.

Formulability burden. Any interface with two distinguishable formulation positions carries an embedded binary distinction. Any set-like candidate defined here has such positions through object diversity, so its own nonterminality supplies distinctional data.

Canonical closure. The completed skeleton has no residual internal choices. Distinction and reading presentations normalize to the canonical completed object; every object and morphism of the skeleton is unique; products, pullbacks, the subobject classifier, and exponentials all have the unique completed object as their classified witness. Remaining questions are classified as raw obstruction, set-like burden, formulability extraction, or semantic adequacy burden.

Route-admitted classification. The paper defines the completion claims admitted by this route as a finite formal interface. Every such claim is classified by case analysis as canonical skeleton, impossible raw topos attempt, set-like burden, or semantic-adequacy burden.

Admitted foundation classification. The paper defines admitted foundation claims as either nontrivial formulable foundations or route-admitted completion claims. Every admitted foundation claim is classified: entry-level foundations by embedded binary distinction, completion-level claims by the route-completion classification.

Scope. The theorem classifies this route's formal admission interfaces. Richer semantics may be formulated, but they enter as additional structures with their own adequacy data; they are not silently counted as already present at the raw or skeleton stages.

Part I. Distinction and completion pressure

1 Setting

The code below starts again from the formal threshold. It imports no results from `Void.lagda.tex`. The imported material is limited to the Agda library components needed for the formal constructions below: universes, propositional equality, the empty type, coproducts, dependent pairs, and negation.

```
-- § Safe fragment: no postulates, no K axiom, no imported Void module.
{-# OPTIONS --safe --without-K #-}

-- § Standalone topos-boundary module.
module VoidTopos where

-- § Imported infrastructure: universes, equality, absurdity, sums, pairs, negation.
open import Agda.Primitive using (Setω)
open import Relation.Binary.PropositionalEquality using (_≡_; refl; sym; trans; cong)
open import Data.Empty using (⊥; ⊥-elim)
open import Data.Sum using (_⊎_; inj₁; inj₂)
open import Data.Product using (Σ; _,_)
open import Relation.Nullary using (¬_)
```

2 Distinction

The first formal object is a binary distinction: a carrier, two named boundary points, a proof that those points do not collapse, and a proof that every point of the carrier is one of them.

```
-- § The two-point normal form has exactly two constructors.
data Two : Set where
 L R : Two

-- § The two boundary names of Two do not collapse.
L≠R : ¬ (L ≡ R)
L≠R ()

-- § The opposite inequality is obtained by symmetry of equality.
R≠L : ¬ (R ≡ L)
R≠L p = L≠R (sym p)

-- § Binary distinction: carrier, left/right boundaries, separation, coverage.
record Distinction : Set₁ where
 field
  S         : Set
  left      : S
  right     : S
  separated : ¬ (left ≡ right)
  cover     : (x : S) → (x ≡ left) ⊎ (x ≡ right)

open Distinction public

-- § The canonical cover of Two is exhaustive by case analysis.
Two-cover : (x : Two) → (x ≡ L) ⊎ (x ≡ R)
Two-cover L = inj₁ refl
Two-cover R = inj₂ refl

-- § The canonical two-point distinction.
Two-distinction : Distinction
Two-distinction = record
 { S         = Two
 ; left      = L
 ; right     = R
 ; separated = L≠R
 ; cover     = Two-cover
 }

-- § The swapped cover changes boundary roles without changing the carrier.
Two-swapped-cover : (x : Two) → (x ≡ R) ⊎ (x ≡ L)
Two-swapped-cover L = inj₂ refl
Two-swapped-cover R = inj₁ refl

-- § Swapped presentation: same Two, opposite left/right boundary names.
Two-distinction-swapped : Distinction
Two-distinction-swapped = record
 { S         = Two
 ; left      = R
 ; right     = L
 ; separated = R≠L
 ; cover     = Two-swapped-cover
 }
```

3 Normal Form

The cover of a distinction determines a map to Two; the two boundary points determine a map back. These maps are inverse in the propositional sense.

```
-- § The cover forces each point into the normal left/right code.
toTwo : (d : Distinction) → S d → Two
```

```
toTwo d x with cover d x
... | inj₁ _ = L
... | inj₂ _ = R

-- § The inverse candidate sends normal points back to the boundary points.
fromTwo : (d : Distinction) → Two → S d
fromTwo d L = left d
fromTwo d R = right d

-- § Left boundary maps to L; the wrong branch contradicts separation.
toTwo-left : (d : Distinction) → toTwo d (left d) ≡ L
toTwo-left d with cover d (left d)
... | inj₁ _ = refl
... | inj₂ left≡right = ⊥-elim (separated d left≡right)

-- § Right boundary maps to R; the wrong branch contradicts separation.
toTwo-right : (d : Distinction) → toTwo d (right d) ≡ R
toTwo-right d with cover d (right d)
... | inj₁ right≡left = ⊥-elim (separated d (sym right≡left))
... | inj₂ _ = refl

-- § Roundtrip from an arbitrary carrier point through Two.
fromTwo-toTwo : (d : Distinction) → (x : S d) → fromTwo d (toTwo d x) ≡ x
fromTwo-toTwo d x with cover d x
... | inj₁ x≡left = sym x≡left
... | inj₂ x≡right = sym x≡right

-- § Roundtrip from Two back through the boundary map.
toTwo-fromTwo : (d : Distinction) → (x : Two) → toTwo d (fromTwo d x) ≡ x
toTwo-fromTwo d L = toTwo-left d
toTwo-fromTwo d R = toTwo-right d
```

Boundary-preserving isomorphism is the equality of presentations used throughout the paper. The normal-form theorem says that every distinction is presentation-equal to the canonical two-point distinction.

```
-- § Boundary-preserving isomorphism of distinction presentations.
record DistinctionIso (d e : Distinction) : Set₁ where
 field
  to       : S d → S e
  from     : S e → S d
  to-from  : (y : S e) → to (from y) ≡ y
  from-to  : (x : S d) → from (to x) ≡ x
  to-left  : to (left d) ≡ left e
  to-right : to (right d) ≡ right e

open DistinctionIso public

-- § Presentation equality is implemented as boundary-preserving isomorphism.
DistinctionPresentationEq : Distinction → Distinction → Set₁
DistinctionPresentationEq = DistinctionIso

-- § Normal form: every distinction is presentation-equal to canonical Two.
two-normal-form : (d : Distinction) → DistinctionPresentationEq d Two-distinction
two-normal-form d = record
 { to       = toTwo d
 ; from     = fromTwo d
 ; to-from  = toTwo-fromTwo d
 ; from-to  = fromTwo-toTwo d
 ; to-left  = toTwo-left d
 ; to-right = toTwo-right d
 }

-- § Reflexivity of presentation equality.
distinction-iso-refl : (d : Distinction) → DistinctionIso d d
distinction-iso-refl d = record
 { to       = λ x → x
 ; from     = λ x → x
 ; to-from  = λ x → refl
```

```
 ; from-to = λ x → refl
 ; to-left   = refl
 ; to-right = refl
 }

-- § Symmetry: reverse the two carrier maps and transport boundary proofs.
distinction-iso-sym : {d e : Distinction} → DistinctionIso d e → DistinctionIso e d
distinction-iso-sym {d} iso = record
 { to       = from iso
 ; from     = to iso
 ; to-from = from-to iso
 ; from-to = to-from iso
 ; to-left   = trans (cong (from iso) (sym (to-left iso))) (from-to iso (left d))
 ; to-right = trans (cong (from iso) (sym (to-right iso))) (from-to iso (right d))
 }

-- § Transitivity: compose the carrier maps and concatenate the boundary proofs.
distinction-iso-trans :
 {d e f : Distinction} →
 DistinctionIso d e → DistinctionIso e f → DistinctionIso d f
distinction-iso-trans iso₁ iso₂ = record
 { to       = λ x → to iso₂ (to iso₁ x)
 ; from     = λ z → from iso₁ (from iso₂ z)
 ; to-from = λ z →
    trans (cong (to iso₂) (to-from iso₁ (from iso₂ z))) (to-from iso₂ z)
 ; from-to = λ x →
    trans (cong (from iso₁) (from-to iso₂ (to iso₁ x))) (from-to iso₁ x)
 ; to-left   = trans (cong (to iso₂) (to-left iso₁)) (to-left iso₂)
 ; to-right = trans (cong (to iso₂) (to-right iso₁)) (to-right iso₂)
 }

-- § The swapped presentation still normalizes to canonical Two.
two-distinction-swapped-normalizes :
 DistinctionPresentationEq Two-distinction-swapped Two-distinction
two-distinction-swapped-normalizes = two-normal-form Two-distinction-swapped

-- § Raw collapse would identify the swapped left boundary R with L.
raw-left-boundary-collapse : Set
raw-left-boundary-collapse = left Two-distinction-swapped ≡ left Two-distinction

-- § That raw collapse is impossible.
raw-left-boundary-collapse-impossible : raw-left-boundary-collapse → ⊥
raw-left-boundary-collapse-impossible = R≠L
```

The last term is the warning sign for completion. The swapped presentation normalizes to the canonical presentation by isomorphism, but its visible left boundary is not the canonical left boundary in the old carrier. Completion must therefore introduce a quotient or skeleton layer; it must not silently read presentation equivalence as naive fieldwise equality of raw records.

4 Skeleton Quotient

The smallest quotient carrier has one displayed object. Every raw distinction embeds into that object. This identifies presentations in a new carrier without asserting that the old records were fieldwise equal.

```
-- § The skeleton has one completed presentation object.
data DistinctionSkeletonObject : Set where
 distinction-skeleton-two : DistinctionSkeletonObject

-- § The sole completed object is represented by canonical Two.
distinction-skeleton-representative : DistinctionSkeletonObject → Distinction
distinction-skeleton-representative distinction-skeleton-two = Two-distinction

-- § Every raw distinction embeds into the same completed object.
distinction-skeleton-normalize : Distinction → DistinctionSkeletonObject
```

```
distinction-skeleton-normalize d = distinction-skeleton-two

-- § The embedding is justified by the normal-form isomorphism.
distinction-skeleton-normalization :
 (d : Distinction) →
 DistinctionPresentationEq d
  (distinction-skeleton-representative (distinction-skeleton-normalize d))
distinction-skeleton-normalization d = two-normal-form d

-- § Quotient completion interface for distinction presentations.
record DistinctionQuotientCompletion : Set₂ where
 field
  dqc-carrier   : Set
  dqc-embed     : Distinction → dqc-carrier
  dqc-canonical : dqc-carrier
  dqc-identify  : (d e : Distinction) →
                  DistinctionPresentationEq d e → dqc-embed d ≡ dqc-embed e
  dqc-normalize : (d : Distinction) → dqc-embed d ≡ dqc-canonical

open DistinctionQuotientCompletion public

-- § Minimal skeleton completion: all presentations identify in one carrier.
distinction-skeleton-completion : DistinctionQuotientCompletion
distinction-skeleton-completion = record
 { dqc-carrier  = DistinctionSkeletonObject
 ; dqc-embed    = distinction-skeleton-normalize
 ; dqc-canonical = distinction-skeleton-two
 ; dqc-identify = λ d e same → refl
 ; dqc-normalize = λ d → refl
 }
```

The completion record is intentionally small. It does not claim that the old presentations have become definitionally equal. It constructs a new carrier in which presentation equality is represented by equality of the completed object. This is the first place where the later topos stage can live without confusing isomorphism with raw record identity.

Part II. The raw stable-reading category

5 Classifier And Stable Readings

A classifier object is only a pair of separated values. A marker into that object determines a truth fiber and hence a visible classified subcarrier. This is classifier-shaped data, not yet a universal subobject classifier theorem.

The point of the reading layer is to keep the normal-form map visible as structure. A distinction does not merely normalize to Two; it also supplies a marker into a two-valued classifier. The raw category below is built from these marked carriers before any quotient has collapsed the presentations.

```
-- § A classifier is a separated pair of classifier values.
record ClassifierObject : Set₁ where
 field
  Ω     : Set
  truth : Ω
  false : Ω
  truth-separated : ¬ (truth ≡ false)

open ClassifierObject public

-- § The canonical classifier is the normal two-point carrier.
TwoClassifier : ClassifierObject
```

```
TwoClassifier = record
{ Ω     = Two
; truth = L
; false = R
; truth-separated = L≠R
}

-- § A marker observes points of A inside the classifier object.
ClassifierMarker : ClassifierObject → Set → Set
ClassifierMarker classifier A = A → Ω classifier

-- § Truth fiber: a point together with proof that its marker is truth.
record TruthFiber (classifier : ClassifierObject)
                  {A : Set}
                  (marker : ClassifierMarker classifier A) : Set where
 constructor truth-fiber
 field
  tf-point : A
  tf-selected : marker tf-point ≡ truth classifier

open TruthFiber public

-- § Classified subcarrier: carrier, inclusion, characteristic marker, selection proof.
record ClassifiedSubcarrier (classifier : ClassifierObject) (A : Set) : Set₁ where
 field
  csc-carrier    : Set
  csc-include    : csc-carrier → A
  csc-characteristic : ClassifierMarker classifier A
  csc-selected   : (x : csc-carrier) →
                   csc-characteristic (csc-include x) ≡ truth classifier

open ClassifiedSubcarrier public

-- § Every marker determines its truth-fiber classified subcarrier.
truth-fiber-classified :
 (classifier : ClassifierObject) →
 {A : Set} →
 ClassifierMarker classifier A → ClassifiedSubcarrier classifier A
truth-fiber-classified classifier marker = record
 { csc-carrier    = TruthFiber classifier marker
 ; csc-include    = tf-point
 ; csc-characteristic = marker
 ; csc-selected = tf-selected
 }

-- § Mono-like subcarrier: displayed mono data plus a built-in characteristic map.
record MonoLikeSubcarrier (classifier : ClassifierObject) (A : Set) : Set₁ where
 field
  ml-carrier    : Set
  ml-include    : ml-carrier → A
  ml-characteristic : ClassifierMarker classifier A
  ml-selected   : (x : ml-carrier) →
                  ml-characteristic (ml-include x) ≡ truth classifier

open MonoLikeSubcarrier public

-- § Truth fibers are mono-like subcarriers for the displayed marker.
truth-fiber-mono-like :
 (classifier : ClassifierObject) →
 {A : Set} →
 ClassifierMarker classifier A → MonoLikeSubcarrier classifier A
truth-fiber-mono-like classifier marker = record
 { ml-carrier    = TruthFiber classifier marker
 ; ml-include    = tf-point
 ; ml-characteristic = marker
 ; ml-selected = tf-selected
 }
```

Stable readings attach classifier-valued observations to a carrier. A distinction induces such a reading by using its normal-form map toTwo as marker.

```
-- § Stable reading: a carrier with compatible left/right readings, marker, boundaries.
record StableReadingInterface (classifier : ClassifierObject) : Set₁ where
 field
  sri-carrier          : Set
  sri-left-reading     : ClassifierMarker classifier sri-carrier
  sri-right-reading    : ClassifierMarker classifier sri-carrier
  sri-marker           : ClassifierMarker classifier sri-carrier
  sri-left-stable      : (x : sri-carrier) → sri-marker x ≡ sri-left-reading x
  sri-right-stable     : (x : sri-carrier) → sri-marker x ≡ sri-right-reading x
  sri-boundary-left    : sri-carrier
  sri-boundary-right   : sri-carrier
  sri-marker-left      : sri-marker sri-boundary-left ≡ truth classifier
  sri-marker-right     : sri-marker sri-boundary-right ≡ false classifier

open StableReadingInterface public

-- § A distinction induces a stable reading by using toTwo as marker.
distinction-stable-reading : Distinction → StableReadingInterface TwoClassifier
distinction-stable-reading d = record
 { sri-carrier          = S d
 ; sri-left-reading     = toTwo d
 ; sri-right-reading    = toTwo d
 ; sri-marker           = toTwo d
 ; sri-left-stable      = λ x → refl
 ; sri-right-stable     = λ x → refl
 ; sri-boundary-left    = left d
 ; sri-boundary-right   = right d
 ; sri-marker-left      = toTwo-left d
 ; sri-marker-right     = toTwo-right d
 }
```

6 Stable Reading Category

Structure maps are marker-preserving functions. They compose, and the identity and associativity laws hold pointwise.

```
-- § Morphisms of raw readings are marker-preserving functions.
record StructureMap (classifier : ClassifierObject)
                    (source target : StableReadingInterface classifier) : Set₁ where
 field
  sm-map  : sri-carrier source → sri-carrier target
  sm-marker-commutes : (x : sri-carrier source) →
                       sri-marker target (sm-map x) ≡ sri-marker source x

open StructureMap public

-- § Morphism equality is pointwise equality of the underlying maps.
record StructureMapEq (classifier : ClassifierObject)
                      (source target : StableReadingInterface classifier)
                      (f g : StructureMap classifier source target) : Set where
 field
  sme-map : (x : sri-carrier source) → sm-map f x ≡ sm-map g x

open StructureMapEq public

-- § Identity morphism preserves the marker by reflexivity.
structure-id :
 (classifier : ClassifierObject) →
 (source : StableReadingInterface classifier) → StructureMap classifier source source
structure-id classifier source = record
 { sm-map = λ x → x
 ; sm-marker-commutes = λ x → refl
 }

-- § Composition preserves markers by transitivity of equality.
structure-compose :
 (classifier : ClassifierObject) →
```

```
  {source middle target : StableReadingInterface classifier} →
  StructureMap classifier middle target →
  StructureMap classifier source middle →
  StructureMap classifier source target
structure-compose classifier g f = record
  { sm-map = λ x → sm-map g (sm-map f x)
  ; sm-marker-commutes = λ x →
      trans (sm-marker-commutes g (sm-map f x)) (sm-marker-commutes f x)
  }

-- § Left identity law holds pointwise.
structure-id-left :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  (f : StructureMap classifier source target) →
  StructureMapEq classifier source target
    (structure-compose classifier (structure-id classifier target) f) f
structure-id-left classifier f = record { sme-map = λ x → refl }

-- § Right identity law holds pointwise.
structure-id-right :
  (classifier : ClassifierObject) →
  {source target : StableReadingInterface classifier} →
  (f : StructureMap classifier source target) →
  StructureMapEq classifier source target
    (structure-compose classifier f (structure-id classifier source)) f
structure-id-right classifier f = record { sme-map = λ x → refl }

-- § Associativity holds pointwise.
structure-assoc :
  (classifier : ClassifierObject) →
  {a b c d : StableReadingInterface classifier} →
  (h : StructureMap classifier c d) →
  (g : StructureMap classifier b c) →
  (f : StructureMap classifier a b) →
  StructureMapEq classifier a d
    (structure-compose classifier h (structure-compose classifier g f))
    (structure-compose classifier (structure-compose classifier h g) f)
structure-assoc classifier h g f = record { sme-map = λ x → refl }

record CategoryData (classifier : ClassifierObject) : Set₂ where
  field
    cat-id      : (source : StableReadingInterface classifier) →
                  StructureMap classifier source source
    cat-compose : {source middle target : StableReadingInterface classifier} →
                  StructureMap classifier middle target →
                  StructureMap classifier source middle →
                  StructureMap classifier source target
    cat-id-left : {source target : StableReadingInterface classifier} →
                  (f : StructureMap classifier source target) →
                  StructureMapEq classifier source target (cat-compose (cat-id target) f) f
    cat-id-right : {source target : StableReadingInterface classifier} →
                   (f : StructureMap classifier source target) →
                   StructureMapEq classifier source target (cat-compose f (cat-id source)) f
    cat-assoc : {a b c d : StableReadingInterface classifier} →
                  (h : StructureMap classifier c d) →
                  (g : StructureMap classifier b c) →
                  (f : StructureMap classifier a b) →
                  StructureMapEq classifier a d
                    (cat-compose h (cat-compose g f))
                    (cat-compose (cat-compose h g) f)

open CategoryData public

-- § Category data for raw stable readings.
stable-reading-category-data :
  (classifier : ClassifierObject) → CategoryData classifier
stable-reading-category-data classifier = record
  { cat-id      = structure-id classifier
  ; cat-compose = structure-compose classifier
  ; cat-id-left = structure-id-left classifier
```

```
; cat-id-right = structure-id-right classifier
; cat-assoc    = structure-assoc classifier
}
```

Thus the raw layer is already categorical. The laws are weak only in the sense that morphism equality is pointwise equality of the underlying maps. No topos claim has been made yet; the following sections test which topos ingredients are actually present at this raw level.

7 Raw Terminal Object

The raw stable-reading category has a genuine terminal object for every classifier. Its carrier is the classifier object itself and its marker is the identity. Every reading maps to it by its marker, and marker preservation makes that map unique up to pointwise equality.

```
-- § Terminal-object interface for the raw stable-reading category.
record StableReadingTerminal (classifier : ClassifierObject) : Set₂ where
 field
  srt-object : StableReadingInterface classifier
  srt-arrow  : (source : StableReadingInterface classifier) →
               StructureMap classifier source srt-object
  srt-unique : {source : StableReadingInterface classifier} →
               (f : StructureMap classifier source srt-object) →
               StructureMapEq classifier source srt-object f (srt-arrow source)

open StableReadingTerminal public

-- § Classifier-as-reading: carrier is Ω and the marker is identity.
classifier-terminal-reading :
 (classifier : ClassifierObject) → StableReadingInterface classifier
classifier-terminal-reading classifier = record
 { sri-carrier         = Ω classifier
 ; sri-left-reading    = λ x → x
 ; sri-right-reading   = λ x → x
 ; sri-marker          = λ x → x
 ; sri-left-stable     = λ x → refl
 ; sri-right-stable    = λ x → refl
 ; sri-boundary-left   = truth classifier
 ; sri-boundary-right  = false classifier
 ; sri-marker-left     = refl
 ; sri-marker-right    = refl
 }

-- § Terminal arrow is forced to be the source marker.
stable-reading-terminal :
 (classifier : ClassifierObject) → StableReadingTerminal classifier
stable-reading-terminal classifier = record
 { srt-object = classifier-terminal-reading classifier
 ; srt-arrow = λ source → record
    { sm-map = sri-marker source
    ; sm-marker-commutes = λ x → refl
    }
 ; srt-unique = λ f → record
    { sme-map = λ x → sm-marker-commutes f x
    }
 }
```

The terminal map is forced: from any reading, the only marker-preserving map into the classifier-as-reading is the marker itself. This is the first topos-relevant structure present at the raw level.

8 Raw Weak Binary Products

The raw stable-reading category also has weak binary products: the canonical product carrier of pairs whose marker values agree. The projection and pairing laws are checked directly. The final

uniqueness law is separated as a boundary because morphism equality into this dependent carrier asks for equality of the stored marker-agreement proofs.

```
-- § Product point: two points whose marker values agree.
record ReadingProductPoint
 (classifier : ClassifierObject)
 (left right : StableReadingInterface classifier) : Set where
 constructor reading-product-point
 field
  rpp-left-point     : sri-carrier left
  rpp-right-point    : sri-carrier right
  rpp-marker-agrees : sri-marker left rpp-left-point ≡ sri-marker right rpp-right-point

open ReadingProductPoint public

-- § Weak binary product interface: projections and pairing, uniqueness left open.
record StableReadingWeakBinaryProduct
 (classifier : ClassifierObject)
 (left right : StableReadingInterface classifier) : Set₁ where
 field
  srwp-object : StableReadingInterface classifier
  srwp-left   : StructureMap classifier srwp-object left
  srwp-right : StructureMap classifier srwp-object right
  srwp-pair  : {source : StableReadingInterface classifier} →
               StructureMap classifier source left →
               StructureMap classifier source right →
               StructureMap classifier source srwp-object
  srwp-pair-left : {source : StableReadingInterface classifier} →
                   (f : StructureMap classifier source left) →
                   (g : StructureMap classifier source right) →
                   StructureMapEq classifier source left
                    (structure-compose classifier srwp-left (srwp-pair f g))
                    f
  srwp-pair-right : {source : StableReadingInterface classifier} →
                    (f : StructureMap classifier source left) →
                    (g : StructureMap classifier source right) →
                    StructureMapEq classifier source right
                     (structure-compose classifier srwp-right (srwp-pair f g))
                     g

open StableReadingWeakBinaryProduct public

-- § Product object uses the common marker value as its marker.
stable-reading-product-object :
 (classifier : ClassifierObject) →
 (left right : StableReadingInterface classifier) →
 StableReadingInterface classifier
stable-reading-product-object classifier left right = record
 { sri-carrier = ReadingProductPoint classifier left right
 ; sri-left-reading = λ point → sri-marker left (rpp-left-point point)
 ; sri-right-reading = λ point → sri-marker left (rpp-left-point point)
 ; sri-marker = λ point → sri-marker left (rpp-left-point point)
 ; sri-left-stable = λ point → refl
 ; sri-right-stable = λ point → refl
 ; sri-boundary-left = reading-product-point
    (sri-boundary-left left)
    (sri-boundary-left right)
    (trans (sri-marker-left left) (sym (sri-marker-left right)))
 ; sri-boundary-right = reading-product-point
    (sri-boundary-right left)
    (sri-boundary-right right)
    (trans (sri-marker-right left) (sym (sri-marker-right right)))
 ; sri-marker-left = sri-marker-left left
 ; sri-marker-right = sri-marker-right left
 }

-- § First projection forgets the right component.
stable-reading-product-left :
 (classifier : ClassifierObject) →
 (left right : StableReadingInterface classifier) →
```

```
 StructureMap classifier (stable-reading-product-object classifier left right) left
stable-reading-product-left classifier left right = record
 { sm-map          = rpp-left-point
 ; sm-marker-commutes = λ point → refl
 }

-- § Second projection forgets the left component and uses marker agreement.
stable-reading-product-right :
 (classifier : ClassifierObject) →
 (left right : StableReadingInterface classifier) →
 StructureMap classifier (stable-reading-product-object classifier left right) right
stable-reading-product-right classifier left right = record
 { sm-map          = rpp-right-point
 ; sm-marker-commutes = λ point → sym (rpp-marker-agrees point)
 }

-- § Pairing combines two marker-preserving maps into one product map.
stable-reading-product-pair :
 (classifier : ClassifierObject) →
 {source left right : StableReadingInterface classifier} →
 StructureMap classifier source left →
 StructureMap classifier source right →
 StructureMap classifier source (stable-reading-product-object classifier left right)
stable-reading-product-pair classifier f g = record
 { sm-map = λ x → reading-product-point
    (sm-map f x)
    (sm-map g x)
    (trans (sm-marker-commutes f x) (sym (sm-marker-commutes g x)))
 ; sm-marker-commutes = λ x → sm-marker-commutes f x
 }

-- § The raw category has weak binary products for every pair of readings.
stable-reading-weak-product :
 (classifier : ClassifierObject) →
 (left right : StableReadingInterface classifier) →
 StableReadingWeakBinaryProduct classifier left right
stable-reading-weak-product classifier left right = record
 { srwp-object = stable-reading-product-object classifier left right
 ; srwp-left = stable-reading-product-left classifier left right
 ; srwp-right = stable-reading-product-right classifier left right
 ; srwp-pair = stable-reading-product-pair classifier
 ; srwp-pair-left = λ f g → record { sme-map = λ x → refl }
 ; srwp-pair-right = λ f g → record { sme-map = λ x → refl }
 }

-- § Package type for all raw weak binary products.
StableReadingWeakBinaryProducts : Set₁
StableReadingWeakBinaryProducts =
 (classifier : ClassifierObject) →
 (left right : StableReadingInterface classifier) →
 StableReadingWeakBinaryProduct classifier left right

stable-reading-weak-products : StableReadingWeakBinaryProducts
stable-reading-weak-products = stable-reading-weak-product

-- § Boundary: final uniqueness of weak products is not supplied here.
StableReadingProductUniquenessBoundary : Set₁
StableReadingProductUniquenessBoundary =
 (classifier : ClassifierObject) →
 (left right : StableReadingInterface classifier) →
 (product : StableReadingWeakBinaryProduct classifier left right) →
 {source : StableReadingInterface classifier} →
 (f : StructureMap classifier source left) →
 (g : StructureMap classifier source right) →
 (h : StructureMap classifier source (srwp-object product)) →
 StructureMapEq classifier source left
  (structure-compose classifier (srwp-left product) h) f →
 StructureMapEq classifier source right
  (structure-compose classifier (srwp-right product) h) g →
 StructureMapEq classifier source (srwp-object product) h (srwp-pair product f g)
```

This construction is deliberately called weak. The projections and pairing maps are present and checked. The final uniqueness principle is recorded as a boundary because equality into the dependent carrier also mentions the proof component witnessing marker agreement. The argument does not silently upgrade this weak product into a proof-relevant universal product.

9 Isomorphism And Univalence Boundary

Identity of stable readings always induces isomorphism. The reverse direction is the univalence-like extension; it is named but not assumed.

```
-- § Stable-reading isomorphism: inverse structure maps up to pointwise equality.
record StableReadingIso (classifier : ClassifierObject)
                        (source target : StableReadingInterface classifier) : Set₁ where
 field
  siso-forward : StructureMap classifier source target
  siso-backward : StructureMap classifier target source
  siso-to-from : StructureMapEq classifier target target
   (structure-compose classifier siso-forward siso-backward)
   (structure-id classifier target)
  siso-from-to : StructureMapEq classifier source source
   (structure-compose classifier siso-backward siso-forward)
   (structure-id classifier source)

open StableReadingIso public

-- § Reflexive isomorphism of a reading with itself.
stable-reading-iso-refl :
 (classifier : ClassifierObject) →
 (source : StableReadingInterface classifier) → StableReadingIso classifier source source
stable-reading-iso-refl classifier source = record
 { siso-forward = structure-id classifier source
 ; siso-backward = structure-id classifier source
 ; siso-to-from  = record { sme-map = λ x → refl }
 ; siso-from-to  = record { sme-map = λ x → refl }
 }

-- § Raw identity of readings always induces stable-reading isomorphism.
stable-reading-identity-to-iso :
 (classifier : ClassifierObject) →
 {source target : StableReadingInterface classifier} →
 source ≡ target → StableReadingIso classifier source target
stable-reading-identity-to-iso classifier refl = stable-reading-iso-refl classifier _

-- § Univalence principle: the reverse direction is named, not assumed.
StableReadingUnivalencePrinciple : ClassifierObject → Set₁
StableReadingUnivalencePrinciple classifier =
 {source target : StableReadingInterface classifier} →
 StableReadingIso classifier source target → source ≡ target

-- § Boundary record: checked identity-to-iso plus the univalent extension slot.
record UnivalenceBoundary (classifier : ClassifierObject) : Set₂ where
 field
  ub-identity-to-iso :
   {source target : StableReadingInterface classifier} →
   source ≡ target → StableReadingIso classifier source target
  ub-univalent-extension : Set₁

open UnivalenceBoundary public

-- § The boundary is populated only by the checked direction.
stable-reading-univalence-boundary :
 (classifier : ClassifierObject) → UnivalenceBoundary classifier
stable-reading-univalence-boundary classifier = record
 { ub-identity-to-iso = stable-reading-identity-to-iso classifier
 ; ub-univalent-extension = StableReadingUnivalencePrinciple classifier
 }
```

Distinction isomorphisms induce isomorphisms of the stable readings that come from distinctions. Thus the normal-form theorem lifts from raw presentations to their displayed reading layer.

```
-- § If a point is propositionally the left boundary, toTwo reads it as L.
toTwo-left-at :
 (d : Distinction) → (x : S d) → x ≡ left d → toTwo d x ≡ L
toTwo-left-at d x x≡left with cover d x
... | inj₁ _ = refl
... | inj₂ x≡right = ⊥-elim (separated d (trans (sym x≡left) x≡right))

-- § If a point is propositionally the right boundary, toTwo reads it as R.
toTwo-right-at :
 (d : Distinction) → (x : S d) → x ≡ right d → toTwo d x ≡ R
toTwo-right-at d x x≡right with cover d x
... | inj₁ x≡left = ⊥-elim (separated d (trans (sym x≡left) x≡right))
... | inj₂ _ = refl

-- § Boundary-preserving isomorphisms commute with normal-form markers.
distinction-iso-marker-commutes :
 {d e : Distinction} →
 (iso : DistinctionIso d e) →
 (x : S d) → toTwo e (to iso x) ≡ toTwo d x
distinction-iso-marker-commutes {d} {e} iso x with cover d x
... | inj₁ x≡left =
 trans
  (toTwo-left-at e (to iso x) (trans (cong (to iso) x≡left) (to-left iso)))
  refl
... | inj₂ x≡right =
 trans
  (toTwo-right-at e (to iso x) (trans (cong (to iso) x≡right) (to-right iso)))
  refl

-- § A distinction isomorphism induces a marker-preserving structure map.
distinction-iso-stable-reading-morphism :
 {d e : Distinction} →
 DistinctionIso d e →
 StructureMap TwoClassifier (distinction-stable-reading d) (distinction-stable-reading e)
distinction-iso-stable-reading-morphism iso = record
 { sm-map = to iso
 ; sm-marker-commutes = distinction-iso-marker-commutes iso
 }

-- § A distinction isomorphism induces an isomorphism of displayed readings.
distinction-iso-stable-reading-iso :
 {d e : Distinction} →
 DistinctionIso d e →
 StableReadingIso TwoClassifier (distinction-stable-reading d) (distinction-stable-reading e)
distinction-iso-stable-reading-iso iso = record
 { siso-forward = distinction-iso-stable-reading-morphism iso
 ; siso-backward = distinction-iso-stable-reading-morphism (distinction-iso-sym iso)
 ; siso-to-from  = record { sme-map = to-from iso }
 ; siso-from-to  = record { sme-map = from-to iso }
 }

-- § Normal form lifts from distinctions to distinction-induced readings.
distinction-stable-reading-normalizes :
 (d : Distinction) →
 StableReadingIso TwoClassifier
  (distinction-stable-reading d)
  (distinction-stable-reading Two-distinction)
distinction-stable-reading-normalizes d =
 distinction-iso-stable-reading-iso (two-normal-form d)
```

The lifting result is the bridge between the original distinction layer and the reading layer. Presentation equality of distinctions induces isomorphism of the displayed readings. This is why the later skeleton quotient can quotient distinction-induced readings without importing the main development.

10 Distinction-Reading Skeleton Quotient

The same skeleton carrier quotients the distinction-induced stable readings. This is not a classification of all stable readings; it is the quotient of the readings obtained from distinctions.

```
-- § Quotient completion interface for distinction-induced readings only.
record DistinctionReadingQuotientCompletion : Set₂ where
 field
  drqc-carrier   : Set
  drqc-reading   : Distinction → StableReadingInterface TwoClassifier
  drqc-embed     : Distinction → drqc-carrier
  drqc-canonical : drqc-carrier
  drqc-identify  : (d e : Distinction) →
                   StableReadingIso TwoClassifier (drqc-reading d) (drqc-reading e) →
                   drqc-embed d ≡ drqc-embed e
  drqc-normalize : (d : Distinction) → drqc-embed d ≡ drqc-canonical

open DistinctionReadingQuotientCompletion public

-- § Skeleton completion of the readings obtained from distinctions.
distinction-reading-skeleton-completion : DistinctionReadingQuotientCompletion
distinction-reading-skeleton-completion = record
 { drqc-carrier   = DistinctionSkeletonObject
 ; drqc-reading   = distinction-stable-reading
 ; drqc-embed     = distinction-skeleton-normalize
 ; drqc-canonical = distinction-skeleton-two
 ; drqc-identify  = λ d e same → refl
 ; drqc-normalize = λ d → refl
 }
```

11 Displayed Classifier Extension

Truth fibers provide the classifier-shaped part of a subobject classifier. For the present mono-like subcarrier data, the characteristic map is already part of the subcarrier. The extension is therefore constructible. This does not classify arbitrary categorical monomorphisms; it classifies exactly the mono-like subcarriers defined here.

```
-- § Displayed universality: a mono-like subcarrier yields its characteristic marker.
SubobjectClassifierUniversalExtension : ClassifierObject → Set₁
SubobjectClassifierUniversalExtension classifier =
{A : Set} → MonoLikeSubcarrier classifier A → ClassifierMarker classifier A

-- § The characteristic marker is already a field of MonoLikeSubcarrier.
mono-like-characteristic :
 (classifier : ClassifierObject) → SubobjectClassifierUniversalExtension classifier
mono-like-characteristic classifier mono-like = ml-characteristic mono-like

-- § Candidate classifier data visible at the raw reading level.
record SubobjectClassifierCandidate (classifier : ClassifierObject) : Set₂ where
 field
  soc-truth-fiber-classified : {A : Set} →
                               ClassifierMarker classifier A →
                               ClassifiedSubcarrier classifier A
  soc-truth-fiber-mono-like  : {A : Set} →
                               ClassifierMarker classifier A →
                               MonoLikeSubcarrier classifier A
  soc-characteristic-visible : {A : Set} →
                               (marker : ClassifierMarker classifier A) →
                               csc-characteristic
                                (soc-truth-fiber-classified marker) ≡ marker
  soc-universal-extension    : SubobjectClassifierUniversalExtension classifier

open SubobjectClassifierCandidate public
```

```
-- § Truth fibers and mono-like characteristic maps give the displayed candidate.
classifier-subobject-candidate :
 (classifier : ClassifierObject) → SubobjectClassifierCandidate classifier
classifier-subobject-candidate classifier = record
 { soc-truth-fiber-classified = truth-fiber-classified classifier
 ; soc-truth-fiber-mono-like = truth-fiber-mono-like classifier
 ; soc-characteristic-visible = λ marker → refl
 ; soc-universal-extension  = mono-like-characteristic classifier
 }
```

This is the second topos-relevant raw structure. It is exactly as strong as the displayed data permits: a mono-like subcarrier already carries its characteristic marker, and the extension returns that marker. The construction does not infer an arbitrary categorical subobject classifier for the raw category from this displayed fact.

Part III. Obstruction and completion

12 Global Pullback Obstruction

An elementary topos structure on the full raw stable-reading category requires pullbacks for arbitrary cospans of marker-preserving maps. The full raw category does not have that structure. The obstruction is already visible over the two-valued classifier: two maps can preserve the same truth marker while landing in different truth points of the apex. Any pullback cone then supplies a left boundary point whose two apex images are equal. That equality is impossible.

The counterexample is finite and explicit. The apex has two different truth-points and two different false-points, all marked by the same two-valued classifier. One map sends truth to the first truth-point; the other sends truth to the second truth-point. Both maps preserve the marker. A pullback cone contains a left boundary point in its object. The two legs of the cone must send that boundary to truth on the left and truth on the right, and the square would identify the two apex images. The apex was built so that this equality is impossible.

```
-- § Pullback cone for a cospan of raw stable-reading morphisms.
record StableReadingPullbackCone
 (classifier : ClassifierObject)
 {left right apex : StableReadingInterface classifier}
 (f : StructureMap classifier left apex)
 (g : StructureMap classifier right apex) : Set₁ where
 field
  srpc-object : StableReadingInterface classifier
  srpc-left   : StructureMap classifier srpc-object left
  srpc-right : StructureMap classifier srpc-object right
  srpc-square : StructureMapEq classifier srpc-object apex
    (structure-compose classifier f srpc-left)
    (structure-compose classifier g srpc-right)

open StableReadingPullbackCone public

-- § Pullback interface: cone, lift, projection laws, uniqueness.
record StableReadingPullback
 (classifier : ClassifierObject)
 {left right apex : StableReadingInterface classifier}
 (f : StructureMap classifier left apex)
 (g : StructureMap classifier right apex) : Set₁ where
 field
  srp-cone : StableReadingPullbackCone classifier f g
  srp-lift : {source : StableReadingInterface classifier} →
             (left-map : StructureMap classifier source left) →
             (right-map : StructureMap classifier source right) →
             StructureMapEq classifier source apex
```

```
          (structure-compose classifier f left-map)
          (structure-compose classifier g right-map) →
          StructureMap classifier source (srpc-object srp-cone)
srp-lift-left : {source : StableReadingInterface classifier} →
                (left-map : StructureMap classifier source left) →
                (right-map : StructureMap classifier source right) →
                (square : StructureMapEq classifier source apex
                  (structure-compose classifier f left-map)
                  (structure-compose classifier g right-map)) →
                StructureMapEq classifier source left
                  (structure-compose classifier (srpc-left srp-cone)
                   (srp-lift left-map right-map square))
                  left-map
srp-lift-right : {source : StableReadingInterface classifier} →
                 (left-map : StructureMap classifier source left) →
                 (right-map : StructureMap classifier source right) →
                 (square : StructureMapEq classifier source apex
                     (structure-compose classifier f left-map)
                     (structure-compose classifier g right-map)) →
                 StructureMapEq classifier source right
                     (structure-compose classifier (srpc-right srp-cone)
                      (srp-lift left-map right-map square))
                     right-map
srp-lift-unique : {source : StableReadingInterface classifier} →
                  (left-map : StructureMap classifier source left) →
                  (right-map : StructureMap classifier source right) →
                  (square : StructureMapEq classifier source apex
                   (structure-compose classifier f left-map)
                   (structure-compose classifier g right-map)) →
                  (h : StructureMap classifier source (srpc-object srp-cone)) →
                  StructureMapEq classifier source left
                   (structure-compose classifier (srpc-left srp-cone) h)
                   left-map →
                  StructureMapEq classifier source right
                   (structure-compose classifier (srpc-right srp-cone) h)
                   right-map →
                  StructureMapEq classifier source (srpc-object srp-cone) h
                   (srp-lift left-map right-map square)

open StableReadingPullback public

-- § Split apex: two truth-points and two false-points.
data SplitApexPoint : Set where
 truth-left-point : SplitApexPoint
 truth-right-point : SplitApexPoint
 false-left-point : SplitApexPoint
 false-right-point : SplitApexPoint

-- § The two truth-points are definitionally distinct constructors.
truth-left-point≠truth-right-point : ¬ (truth-left-point ≡ truth-right-point)
truth-left-point≠truth-right-point ()

-- § The marker identifies both truth-points as truth, both false-points as false.
split-apex-marker : SplitApexPoint → Two
split-apex-marker truth-left-point = L
split-apex-marker truth-right-point = L
split-apex-marker false-left-point = R
split-apex-marker false-right-point = R

-- § The two-valued classifier as a stable reading.
two-classifier-reading : StableReadingInterface TwoClassifier
two-classifier-reading = classifier-terminal-reading TwoClassifier

-- § The split apex is a stable reading over the two-valued classifier.
split-truth-apex : StableReadingInterface TwoClassifier
split-truth-apex = record
 { sri-carrier       = SplitApexPoint
 ; sri-left-reading  = split-apex-marker
 ; sri-right-reading = split-apex-marker
 ; sri-marker        = split-apex-marker
 ; sri-left-stable   = λ x → refl
```

```
 ; sri-right-stable    = λ x → refl
 ; sri-boundary-left = truth-left-point
 ; sri-boundary-right = false-left-point
 ; sri-marker-left    = refl
 ; sri-marker-right   = refl
 }

-- § Left leg of the cospan chooses the left truth/false copies.
split-left-map :
 StructureMap TwoClassifier two-classifier-reading split-truth-apex
split-left-map = record
 { sm-map = λ
    { L → truth-left-point
    ; R → false-left-point
    }
 ; sm-marker-commutes = λ
    { L → refl
    ; R → refl
    }
 }

-- § Right leg of the cospan chooses the right truth/false copies.
split-right-map :
 StructureMap TwoClassifier two-classifier-reading split-truth-apex
split-right-map = record
 { sm-map = λ
    { L → truth-right-point
    ; R → false-right-point
    }
 ; sm-marker-commutes = λ
    { L → refl
    ; R → refl
    }
 }

-- § If the left leg sees L, it lands at the left truth-point.
split-left-map-at :
 (x : Two) → x ≡ L → sm-map split-left-map x ≡ truth-left-point
split-left-map-at L refl = refl
split-left-map-at R ()

-- § If the right leg sees L, it lands at the right truth-point.
split-right-map-at :
 (x : Two) → x ≡ L → sm-map split-right-map x ≡ truth-right-point
split-right-map-at L refl = refl
split-right-map-at R ()

-- § A cone's left projection sends the cone-left boundary to L.
split-cone-left-projection-left :
 (cone : StableReadingPullbackCone TwoClassifier split-left-map split-right-map) →
 sm-map (srpc-left cone) (sri-boundary-left (srpc-object cone)) ≡ L
split-cone-left-projection-left cone =
 trans
  (sm-marker-commutes
    (srpc-left cone)
    (sri-boundary-left (srpc-object cone)))
  (sri-marker-left (srpc-object cone))

-- § A cone's right projection also sends the cone-left boundary to L.
split-cone-right-projection-left :
 (cone : StableReadingPullbackCone TwoClassifier split-left-map split-right-map) →
 sm-map (srpc-right cone) (sri-boundary-left (srpc-object cone)) ≡ L
split-cone-right-projection-left cone =
 trans
  (sm-marker-commutes
    (srpc-right cone)
    (sri-boundary-left (srpc-object cone)))
  (sri-marker-left (srpc-object cone))

-- § No cone exists: the square would identify the two distinct truth-points.
split-truth-cospan-has-no-cone :
```

```
 StableReadingPullbackCone TwoClassifier split-left-map split-right-map → ⊥
split-truth-cospan-has-no-cone cone =
 truth-left-point≠truth-right-point
  (trans
    (sym
     (split-left-map-at
      (sm-map (srpc-left cone) (sri-boundary-left (srpc-object cone)))
      (split-cone-left-projection-left cone)))
    (trans
     (sme-map (srpc-square cone) (sri-boundary-left (srpc-object cone)))
     (split-right-map-at
      (sm-map (srpc-right cone) (sri-boundary-left (srpc-object cone)))
      (split-cone-right-projection-left cone))))
```

The obstruction is stronger than a missing construction. It is a contradiction from the assumption that the displayed cospan has even a pullback cone. Consequently the full raw category cannot be upgraded to an elementary topos by supplying omitted proof terms.

13 Topos Boundary

The full raw topos requirements cannot be jointly supplied at the raw level. The terminal object is present, but global pullbacks are not available for the whole raw category. The boundary record therefore separates the checked data, the formal obstruction, and the named requirements whose conjunction cannot be supplied.

```
-- § Global pullbacks would assign a pullback to every raw cospan.
StableReadingGlobalPullbacks : Set₁
StableReadingGlobalPullbacks =
 (classifier : ClassifierObject) →
 (left right apex : StableReadingInterface classifier) →
 (f : StructureMap classifier left apex) →
 (g : StructureMap classifier right apex) →
 StableReadingPullback classifier {left} {right} {apex} f g

-- § Global pullbacks contradict the split-truth cospan.
raw-global-pullbacks-impossible : StableReadingGlobalPullbacks → ⊥
raw-global-pullbacks-impossible global-pullbacks =
 split-truth-cospan-has-no-cone
  (srp-cone
   (global-pullbacks
    TwoClassifier
    two-classifier-reading
    two-classifier-reading
    split-truth-apex
    split-left-map
    split-right-map))

-- § Displayed classifier extension for all classifiers.
StableReadingSubobjectClassifierUniversality : Set₁
StableReadingSubobjectClassifierUniversality =
 (classifier : ClassifierObject) → SubobjectClassifierUniversalExtension classifier

-- § The displayed classifier extension is implemented by mono-like characteristic maps.
stable-reading-subobject-classifier-universality :
 StableReadingSubobjectClassifierUniversality
stable-reading-subobject-classifier-universality = mono-like-characteristic

-- § Exponentials remain a named raw-level requirement.
StableReadingExponentials : Set₂
StableReadingExponentials =
 (classifier : ClassifierObject) →
 StableReadingInterface classifier →
 StableReadingInterface classifier →
 Set₁
```

```
-- § Boundary map: checked raw data, obstruction, quotient data, and unmet requirements.
record ToposBoundaryMap : Setω where
 field
  tb-category-data     : (classifier : ClassifierObject) → CategoryData classifier
  tb-terminal-reading : (classifier : ClassifierObject) →
                          StableReadingTerminal classifier
  tb-weak-products    : StableReadingWeakBinaryProducts
  tb-product-uniqueness-boundary : Set₁
  tb-subobject-candidate : (classifier : ClassifierObject) →
                          SubobjectClassifierCandidate classifier
  tb-distinction-quotient : DistinctionQuotientCompletion
  tb-reading-quotient : DistinctionReadingQuotientCompletion
  tb-univalence-boundary : (classifier : ClassifierObject) →
                          UnivalenceBoundary classifier
  tb-required-global-pullbacks : Set₁
  tb-global-pullbacks-obstruction : StableReadingGlobalPullbacks → ⊥
  tb-subobject-universal : StableReadingSubobjectClassifierUniversality
  tb-required-exponentials : Set₂
  tb-required-stable-univalence : (classifier : ClassifierObject) → Set₁

open ToposBoundaryMap public

-- § The boundary map records exactly what the raw stage supplies and blocks.
topos-boundary-map : ToposBoundaryMap
topos-boundary-map = record
 { tb-category-data                = stable-reading-category-data
 ; tb-terminal-reading             = stable-reading-terminal
 ; tb-weak-products                = stable-reading-weak-products
 ; tb-product-uniqueness-boundary = StableReadingProductUniquenessBoundary
 ; tb-subobject-candidate          = classifier-subobject-candidate
 ; tb-distinction-quotient         = distinction-skeleton-completion
 ; tb-reading-quotient             = distinction-reading-skeleton-completion
 ; tb-univalence-boundary          = stable-reading-univalence-boundary
 ; tb-required-global-pullbacks    = StableReadingGlobalPullbacks
 ; tb-global-pullbacks-obstruction = raw-global-pullbacks-impossible
 ; tb-subobject-universal          = stable-reading-subobject-classifier-universality
 ; tb-required-exponentials        = StableReadingExponentials
 ; tb-required-stable-univalence   = StableReadingUnivalencePrinciple
 }

-- § Hypothetical requirements for a full raw topos package.
record ToposCompletionRequirements : Setω where
 field
  tcr-distinction-quotient : DistinctionQuotientCompletion
  tcr-reading-quotient      : DistinctionReadingQuotientCompletion
  tcr-global-pullbacks      : StableReadingGlobalPullbacks
  tcr-subobject-universal   : StableReadingSubobjectClassifierUniversality
  tcr-exponentials          : StableReadingExponentials
  tcr-stable-univalence     : (classifier : ClassifierObject) →
                              StableReadingUnivalencePrinciple classifier

open ToposCompletionRequirements public

-- § Such a raw package is impossible because it contains global pullbacks.
topos-completion-requirements-impossible :
 ToposCompletionRequirements → ⊥
topos-completion-requirements-impossible requirements =
 raw-global-pullbacks-impossible (tcr-global-pullbacks requirements)

-- § Package form: boundary plus impossible raw requirements.
record VoidToposCompletionPackage : Setω where
 field
  vtcp-boundary  : ToposBoundaryMap
  vtcp-requirements : ToposCompletionRequirements

open VoidToposCompletionPackage public

-- § Any supplied requirements would package with the boundary, but cannot exist.
void-topos-completion-package :
 ToposCompletionRequirements → VoidToposCompletionPackage
void-topos-completion-package requirements = record
```

```
{ vtcp-boundary = topos-boundary-map
; vtcp-requirements = requirements
}
```

The requirement record remains useful even though it cannot be inhabited for the full raw category. It states exactly what a raw topos package would have to contain, and topos-completion-requirements-impossible shows that such a package cannot be supplied at this level. This is the formal boundary: the topos package is not first. It is blocked before completion and appears only after completion.

14 Completed Skeleton Category

The quotient carrier has only one displayed object. Its categorical completion is therefore the terminal category: one object, one morphism, and all laws by reflexivity. This is not a topos claim about the raw stable-reading category. It is a topos claim about the completed skeleton carrier obtained after quotienting presentation data.

```
-- § The completed skeleton has a singleton morphism witness.
record One : Set where
 constructor one

-- § Small category interface used for the completed endpoint.
record SmallCategory : Set₁ where
 field
  ObjC     : Set
  HomC     : ObjC → ObjC → Set
  idC      : {A : ObjC} → HomC A A
  compC    : {A B C : ObjC} → HomC B C → HomC A B → HomC A C
  id-leftC  : {A B : ObjC} → (f : HomC A B) → compC idC f ≡ f
  id-rightC : {A B : ObjC} → (f : HomC A B) → compC f idC ≡ f
  assocC   : {A B C D : ObjC} →
             (h : HomC C D) →
             (g : HomC B C) →
             (f : HomC A B) →
             compC h (compC g f) ≡ compC (compC h g) f

open SmallCategory public

-- § The skeleton category: one completed object and one morphism everywhere.
skeleton-category : SmallCategory
skeleton-category = record
 { ObjC    = DistinctionSkeletonObject
 ; HomC    = λ A B → One
 ; idC     = one
 ; compC   = λ g f → one
 ; id-leftC  = λ f → refl
 ; id-rightC = λ f → refl
 ; assocC  = λ h g f → refl
 }

-- § Embedding of a raw distinction into the completed category object.
distinction-to-completed-object : Distinction → SmallCategory.ObjC skeleton-category
distinction-to-completed-object = dqc-embed distinction-skeleton-completion

-- § Every raw distinction lands at the canonical completed object.
distinction-to-completed-object-normalizes :
 (d : Distinction) →
 distinction-to-completed-object d ≡ dqc-canonical distinction-skeleton-completion
distinction-to-completed-object-normalizes =
 dqc-normalize distinction-skeleton-completion
```

The skeleton category is not introduced to add hidden structure. It is the categorical reading of the already constructed quotient carrier. As soon as the carrier has one completed object, there is a unique morphism between any two completed objects, because there are no two distinct completed objects left to distinguish.

15 Finite Limits In The Completion

The terminal category has terminal object, binary products, and pullbacks. The proofs are short because every relevant hom-type is contractible. This is the compact finite-limit core needed by the topos reading of the completed skeleton.

```
-- § Terminal object in an arbitrary small category.
record TerminalObject (category : SmallCategory) : Set₁ where
 open SmallCategory category renaming (ObjC to Obj; HomC to Hom)
 field
  terminal-object : Obj
  terminal-arrow  : (A : Obj) → Hom A terminal-object
  terminal-unique : {A : Obj} →
                    (f : Hom A terminal-object) → f ≡ terminal-arrow A

open TerminalObject public

-- § Binary product in an arbitrary small category.
record BinaryProduct
 (category : SmallCategory)
 (A B : SmallCategory.ObjC category) : Set₁ where
 open SmallCategory category renaming (ObjC to Obj; HomC to Hom; compC to comp)
 field
  product-object : Obj
  product-pr1    : Hom product-object A
  product-pr2    : Hom product-object B
  product-pair   : {X : Obj} →
                   Hom X A → Hom X B → Hom X product-object
  product-left   : {X : Obj} →
                   (f : Hom X A) →
                   (g : Hom X B) →
                   comp product-pr1 (product-pair f g) ≡ f
  product-right  : {X : Obj} →
                   (f : Hom X A) →
                   (g : Hom X B) →
                   comp product-pr2 (product-pair f g) ≡ g
  product-unique : {X : Obj} →
                   (f : Hom X A) →
                   (g : Hom X B) →
                   (h : Hom X product-object) →
                   comp product-pr1 h ≡ f →
                   comp product-pr2 h ≡ g →
                   h ≡ product-pair f g

open BinaryProduct public

-- § Binary products for all pairs of objects.
record HasBinaryProducts (category : SmallCategory) : Set₁ where
 open SmallCategory category renaming (ObjC to Obj)
 field
  product-of : (A B : Obj) → BinaryProduct category A B

open HasBinaryProducts public

-- § Pullback of a cospan in an arbitrary small category.
record Pullback
 (category : SmallCategory)
 {A B Z : SmallCategory.ObjC category}
 (f : HomC category A Z)
 (g : HomC category B Z) : Set₁ where
 open SmallCategory category renaming (ObjC to Obj; HomC to Hom; compC to comp)
 field
  pullback-object : Obj
  pullback-left   : Hom pullback-object A
  pullback-right  : Hom pullback-object B
  pullback-square : comp f pullback-left ≡ comp g pullback-right
  pullback-lift   : {X : Obj} →
                    (u : Hom X A) →
                    (v : Hom X B) →
```

```
                    comp f u ≡ comp g v →
                    Hom X pullback-object
  pullback-lift-left : {X : Obj} →
                    (u : Hom X A) →
                    (v : Hom X B) →
                    (p : comp f u ≡ comp g v) →
                    comp pullback-left (pullback-lift u v p) ≡ u
  pullback-lift-right : {X : Obj} →
                     (u : Hom X A) →
                     (v : Hom X B) →
                     (p : comp f u ≡ comp g v) →
                     comp pullback-right (pullback-lift u v p) ≡ v
  pullback-lift-unique : {X : Obj} →
                       (u : Hom X A) →
                       (v : Hom X B) →
                       (p : comp f u ≡ comp g v) →
                       (h : Hom X pullback-object) →
                       comp pullback-left h ≡ u →
                       comp pullback-right h ≡ v →
                       h ≡ pullback-lift u v p

open Pullback public

-- § Pullbacks for all cospans in a small category.
record HasPullbacks (category : SmallCategory) : Set₁ where
 open SmallCategory category renaming (ObjC to Obj; HomC to Hom)
 field
  pullback-of : {A B Z : Obj} →
                (f : Hom A Z) →
                (g : Hom B Z) →
                Pullback category f g

open HasPullbacks public

-- § The one-object skeleton has a terminal object.
skeleton-terminal : TerminalObject skeleton-category
skeleton-terminal = record
 { terminal-object = distinction-skeleton-two
 ; terminal-arrow = λ A → one
 ; terminal-unique = λ f → refl
 }

-- § Any two skeleton objects have the unique binary product.
skeleton-product :
 (A B : SmallCategory.ObjC skeleton-category) → BinaryProduct skeleton-category A B
skeleton-product A B = record
 { product-object = distinction-skeleton-two
 ; product-pr1   = one
 ; product-pr2   = one
 ; product-pair  = λ f g → one
 ; product-left  = λ f g → refl
 ; product-right = λ f g → refl
 ; product-unique = λ f g h left-proof right-proof → refl
 }

-- § Hence the skeleton category has binary products.
skeleton-products : HasBinaryProducts skeleton-category
skeleton-products = record
 { product-of = skeleton-product
 }

-- § Any skeleton cospan has the unique pullback.
skeleton-pullback :
 {A B Z : SmallCategory.ObjC skeleton-category} →
 (f : HomC skeleton-category A Z) →
 (g : HomC skeleton-category B Z) →
 Pullback skeleton-category {A} {B} {Z} f g
skeleton-pullback {A} {B} {Z} f g = record
 { pullback-object   = distinction-skeleton-two
 ; pullback-left     = one
 ; pullback-right    = one
```

```
 ; pullback-square  = refl
 ; pullback-lift        = λ u v p → one
 ; pullback-lift-left   = λ u v p → refl
 ; pullback-lift-right = λ u v p → refl
 ; pullback-lift-unique = λ u v p h left-proof right-proof → refl
 }

-- § Hence the skeleton category has all pullbacks.
skeleton-pullbacks : HasPullbacks skeleton-category
skeleton-pullbacks = record
 { pullback-of = λ {A} {B} {Z} f g → skeleton-pullback {A} {B} {Z} f g
 }
```

At the completion stage, the finite-limit problem has changed shape. The raw category failed because a nontrivial apex could split two truth-points. The skeleton category has no such split left. Every object is the completed object, every morphism is the unique morphism, and the universal clauses reduce to equality of that unique morphism.

16 Subobjects And Exponentials In The Completion

The terminal category also has a subobject classifier and exponentials. There is only one subobject up to the completed equality, and every exponential object is the unique completed object. The universal clauses again reduce to reflexivity because there is only one morphism wherever a morphism can be written.

```
-- § Mono predicate for morphisms in a small category.
record IsMono
 (category : SmallCategory)
 {A B : SmallCategory.ObjC category}
 (morphism : HomC category A B) : Set₁ where
 open SmallCategory category renaming (ObjC to Obj; HomC to Hom; compC to comp)
 field
  mono-cancel : {X : Obj} →
                (f g : Hom X A) →
                comp morphism f ≡ comp morphism g → f ≡ g

open IsMono public

-- § Pullback square used by the categorical subobject classifier.
record PullbackSquare
 (category : SmallCategory)
 {S A B Z : SmallCategory.ObjC category}
 (left-map : HomC category S A)
 (right-map : HomC category S B)
 (top-map : HomC category A Z)
 (bottom-map : HomC category B Z) : Set₁ where
 open SmallCategory category renaming (ObjC to Obj; HomC to Hom; compC to comp)
 field
  square-commutes : comp top-map left-map ≡ comp bottom-map right-map
  square-lift        : {X : Obj} →
                       (u : Hom X A) →
                       (v : Hom X B) →
                       comp top-map u ≡ comp bottom-map v →
                       Hom X S
  square-lift-left : {X : Obj} →
                     (u : Hom X A) →
                     (v : Hom X B) →
                     (p : comp top-map u ≡ comp bottom-map v) →
                     comp left-map (square-lift u v p) ≡ u
  square-lift-right : {X : Obj} →
                      (u : Hom X A) →
                      (v : Hom X B) →
                      (p : comp top-map u ≡ comp bottom-map v) →
                      comp right-map (square-lift u v p) ≡ v
  square-lift-unique : {X : Obj} →
```

```
                    (u : Hom X A) →
                    (v : Hom X B) →
                    (p : comp top-map u ≡ comp bottom-map v) →
                    (h : Hom X S) →
                    comp left-map h ≡ u →
                    comp right-map h ≡ v →
                    h ≡ square-lift u v p

open PullbackSquare public

-- § Subobject classifier interface in a small category.
record SubobjectClassifierC
 (category : SmallCategory)
 (terminal : TerminalObject category) : Set₁ where
 open SmallCategory category renaming (ObjC to Obj; HomC to Hom)
 open TerminalObject terminal renaming
  (terminal-object to top; terminal-arrow to terminate)
 field
  classifier-object : Obj
  true-map          : Hom top classifier-object
  characteristic    : {Sub Base : Obj} →
                      (morphism : Hom Sub Base) →
                      IsMono category morphism →
                      Hom Base classifier-object
  characteristic-square : {Sub Base : Obj} →
                          (morphism : Hom Sub Base) →
                          (mono : IsMono category morphism) →
                          PullbackSquare category
                           morphism
                           (terminate Sub)
                           (characteristic morphism mono)
                           true-map

open SubobjectClassifierC public

-- § Exponential object interface in a small category.
record Exponential
 (category : SmallCategory)
 (products : HasBinaryProducts category)
 (A B : SmallCategory.ObjC category) : Set₁ where
 open SmallCategory category renaming (ObjC to Obj; HomC to Hom; compC to comp)
 open HasBinaryProducts products renaming (product-of to prod)
 field
  exp-object    : Obj
  exp-eval      : Hom
                   (product-object (prod exp-object A))
                   B
  exp-transpose : {X : Obj} →
                  Hom (product-object (prod X A)) B →
                  Hom X exp-object
  exp-beta      : {X : Obj} →
                  (h : Hom (product-object (prod X A)) B) →
                  comp exp-eval
                   (product-pair (prod exp-object A)
                    (comp (exp-transpose h)
                     (product-pr1 (prod X A)))
                    (product-pr2 (prod X A))) ≡ h
  exp-eta       : {X : Obj} →
                  (h : Hom (product-object (prod X A)) B) →
                  (k : Hom X exp-object) →
                  comp exp-eval
                   (product-pair (prod exp-object A)
                    (comp k (product-pr1 (prod X A)))
                    (product-pr2 (prod X A))) ≡ h →
                  k ≡ exp-transpose h

open Exponential public

-- § Exponentials for all pairs of objects.
record HasExponentials
 (category : SmallCategory)
```

```
 (products : HasBinaryProducts category) : Set₁ where
 open SmallCategory category renaming (ObjC to Obj)
 field
  exponential-of : (A B : Obj) → Exponential category products A B

open HasExponentials public

-- § Every skeleton morphism is mono because all hom-sets are singleton.
skeleton-mono :
{A B : SmallCategory.ObjC skeleton-category} →
(morphism : HomC skeleton-category A B) →
IsMono skeleton-category {A} {B} morphism
skeleton-mono {A} {B} morphism = record
{ mono-cancel = λ f g p → refl
}

-- § The one-object skeleton carries its subobject classifier directly.
skeleton-subobject-classifier :
 SubobjectClassifierC skeleton-category skeleton-terminal
skeleton-subobject-classifier = record
{ classifier-object = distinction-skeleton-two
; true-map          = one
; characteristic    = λ morphism mono → one
; characteristic-square = λ morphism mono → record
   { square-commutes = refl
   ; square-lift          = λ u v p → one
   ; square-lift-left     = λ u v p → refl
   ; square-lift-right    = λ u v p → refl
   ; square-lift-unique = λ u v p h left-proof right-proof → refl
   }
}

-- § The skeleton exponential is the unique object with the unique evaluator.
skeleton-exponential :
 (A B : SmallCategory.ObjC skeleton-category) →
 Exponential skeleton-category skeleton-products A B
skeleton-exponential A B = record
{ exp-object    = distinction-skeleton-two
; exp-eval      = one
; exp-transpose = λ h → one
; exp-beta      = λ h → refl
; exp-eta       = λ h k p → refl
}

-- § Hence the skeleton category has exponentials.
skeleton-exponentials : HasExponentials skeleton-category skeleton-products
skeleton-exponentials = record
{ exponential-of = skeleton-exponential
}
```

The same collapse supplies exponentials and the subobject classifier. This is not a rich internal universe of subobjects; it is the terminal category. The point is exact location: the elementary-topos axioms hold at the quotient-completed skeleton stage, not at the full raw reading stage.

17 The Completion Topos Stage

Putting the pieces together gives the central theorem of this construction: the skeleton completion of distinction presentations carries an elementary topos structure. The completion is not an implicit technical step; it is the explicit topos stage. The raw stable-reading category carries real structure, but its full topos requirements are blocked by the pullback obstruction above.

```
-- § Elementary-topos package used for the completed endpoint.
record ElementaryTopos (category : SmallCategory) : Set₁ where
 field
  et-terminal       : TerminalObject category
```

```
  et-products       : HasBinaryProducts category
  et-pullbacks      : HasPullbacks category
  et-exponentials   : HasExponentials category et-products
  et-subobject-classifier : SubobjectClassifierC category et-terminal

open ElementaryTopos public

-- § The completed skeleton category satisfies the elementary-topos data.
skeleton-elementary-topos : ElementaryTopos skeleton-category
skeleton-elementary-topos = record
 { et-terminal       = skeleton-terminal
 ; et-products       = skeleton-products
 ; et-pullbacks      = skeleton-pullbacks
 ; et-exponentials   = skeleton-exponentials
 ; et-subobject-classifier = skeleton-subobject-classifier
 }

-- § Explicit completion stage: quotient data, category, and topos proof.
record CompletionToposStage : Setω where
 field
  cts-distinction-quotient : DistinctionQuotientCompletion
  cts-reading-quotient  : DistinctionReadingQuotientCompletion
  cts-category          : SmallCategory
  cts-terminal          : TerminalObject cts-category
  cts-products          : HasBinaryProducts cts-category
  cts-pullbacks         : HasPullbacks cts-category
  cts-exponentials      : HasExponentials cts-category cts-products
  cts-subobject-classifier : SubobjectClassifierC cts-category cts-terminal
  cts-topos             : ElementaryTopos cts-category

open CompletionToposStage public

-- § The quotient/skeleton completion is the topos endpoint.
completion-topos-stage : CompletionToposStage
completion-topos-stage = record
 { cts-distinction-quotient = distinction-skeleton-completion
 ; cts-reading-quotient = distinction-reading-skeleton-completion
 ; cts-category         = skeleton-category
 ; cts-terminal         = skeleton-terminal
 ; cts-products         = skeleton-products
 ; cts-pullbacks        = skeleton-pullbacks
 ; cts-exponentials     = skeleton-exponentials
 ; cts-subobject-classifier = skeleton-subobject-classifier
 ; cts-topos            = skeleton-elementary-topos
 }
```

This record is the explicit topos stage of the construction. It keeps the two quotients, the completed category, the finite-limit data, exponentials, the subobject classifier, and the elementary-topos package in one object. The final theorem below does not leave the role of completion implicit; it names that stage directly.

```
-- § Completion theorem package focused on the skeleton topos endpoint.
record VoidSkeletonToposCompletion : Setω where
 field
  vst-topos-stage       : CompletionToposStage
  vst-distinction-quotient : DistinctionQuotientCompletion
  vst-reading-quotient  : DistinctionReadingQuotientCompletion
  vst-category          : SmallCategory
  vst-topos             : ElementaryTopos vst-category

open VoidSkeletonToposCompletion public

-- § The completed skeleton carries the elementary-topos structure.
void-skeleton-topos-completion : VoidSkeletonToposCompletion
void-skeleton-topos-completion = record
 { vst-topos-stage       = completion-topos-stage
 ; vst-distinction-quotient = distinction-skeleton-completion
 ; vst-reading-quotient = distinction-reading-skeleton-completion
 ; vst-category         = skeleton-category
 ; vst-topos            = skeleton-elementary-topos
 }
```

18 Scope Conditions

The theorem is precise in three ways. First, it defines a formal stability interface and proves results about the marker and reading structures that inhabit that interface. Second, it proves the minimal skeleton completion route; the topos stage obtained there is terminal and serves as the first certified endpoint. Third, it turns richer quotients and semantic adequacy into explicit extension burdens. Such systems may extend the construction, but in this route they come after the distinctional and quotient structure already displayed here.

The exhaustive classifications proved later are therefore interface-bound. They classify the nontrivial formulable foundation claims and completion claims admitted by the displayed interfaces. A richer semantic account can enter the discussion only by supplying the additional data named by those interfaces.

```
-- § Semantic adequacy is named as an external problem, not proved here.
StableReadingSemanticAdequacy : Set₂
StableReadingSemanticAdequacy =
 (classifier : ClassifierObject) →
 StableReadingInterface classifier →
 Set₁

-- § A generic richer-completion problem is named, not taken as primitive.
record RicherCompletionCandidate : Set₂ where
 field
  rcc-object-carrier : Set
  rcc-reading-embed : StableReadingInterface TwoClassifier → rcc-object-carrier
  rcc-category       : SmallCategory
  rcc-topos          : ElementaryTopos rcc-category

open RicherCompletionCandidate public

-- § Scope conditions state exactly what the theorem does and does not close.
record VoidToposScopeConditions : Setω where
 field
  csc-semantic-adequacy-problem : Set₂
  csc-minimal-completion-stage  : CompletionToposStage
  csc-richer-completion-problem : Set₂

open VoidToposScopeConditions public

-- § The scope record names the burden carried by richer completions.
void-topos-scope-conditions : VoidToposScopeConditions
void-topos-scope-conditions = record
 { csc-semantic-adequacy-problem = StableReadingSemanticAdequacy
 ; csc-minimal-completion-stage = completion-topos-stage
 ; csc-richer-completion-problem = RicherCompletionCandidate
 }
```

These scope conditions are part of the force of the statement. They make the theorem a route theorem rather than an informal survey of every possible semantics. The displayed route closes the minimal completion stage and names the stronger questions as structured burdens. The later admitted-foundation and route-completion classifications make that position formal: a foundation claim first enters as nontrivial and formulable, and a richer set-like quotient extending this route is assessed by the burdens under which it carries the distinctional structure it uses.

Part IV. Set-like completion burden

19 Why The Minimal Topos Is Not The Set-Like Goal

The completed skeleton is an elementary topos, but it is not the set-like universe one would need for set-level mathematical formulation. The reason is simple and formal: the skeleton has one

completed object. In the displayed set-like sense used here, object diversity is required: there must be at least two distinguishable objects, otherwise object-level distinction has already collapsed.

This locates the standard set-like expectation inside the route. The terminal skeleton is the first certified topos threshold. A set-like stage is a further structured completion above that threshold, where object diversity and semantic adequacy have to be supplied explicitly.

```
-- § Object diversity: the category has two distinct objects.
record HasDistinctObjects (category : SmallCategory) : Set₁ where
 field
  first-object     : SmallCategory.ObjC category
  second-object : SmallCategory.ObjC category
  objects-separated : ¬ (first-object ≡ second-object)

open HasDistinctObjects public

-- § The skeleton completion has no pair of distinct objects.
skeleton-has-no-distinct-objects :
 HasDistinctObjects skeleton-category → ⊥
skeleton-has-no-distinct-objects record
 { first-object = distinction-skeleton-two
 ; second-object = distinction-skeleton-two
 ; objects-separated = separated
 } = separated refl

-- § The minimal completion stage is therefore not set-like by object diversity.
minimal-completion-not-set-like :
 HasDistinctObjects (cts-category completion-topos-stage) → ⊥
minimal-completion-not-set-like = skeleton-has-no-distinct-objects
```

20 A Set-Like Candidate Burden

A richer completion satisfying the displayed object-diversity condition does more than satisfy the elementary-topos package of the terminal skeleton: it is nonterminal. The candidate interface below records the stronger set-like burden package: it hosts the reading data by an explicit embedding, lets points detect maps, carries an arithmetic object, and provides a controlled section principle for cover-like maps. The next section constructs the free syntactic inhabitant of this interface. Thus the interface first names the obligations, and the free completion then shows that the displayed obligations are jointly inhabited on the route's syntactic completion stage.

```
-- § Point-separation principle for morphisms.
record PointsSeparateMorphisms
 (category : SmallCategory)
 (terminal : TerminalObject category) : Set₁ where
 field
  points-separate :
   {A B : SmallCategory.ObjC category} →
   (f g : SmallCategory.HomC category A B) →
   ((point : SmallCategory.HomC category
             (TerminalObject.terminal-object terminal) A) →
     SmallCategory.compC category f point ≡
     SmallCategory.compC category g point) →
   f ≡ g

open PointsSeparateMorphisms public

-- § Natural-numbers-object obligation for a set-like completion.
record NaturalNumbersObject
 (category : SmallCategory)
 (terminal : TerminalObject category) : Set₁ where
 field
  nno-object : SmallCategory.ObjC category
  nno-zero : SmallCategory.HomC category
```

```
              (TerminalObject.terminal-object terminal)
              nno-object
  nno-succ : SmallCategory.HomC category nno-object nno-object
  nno-rec  : {X : SmallCategory.ObjC category} →
             SmallCategory.HomC category
              (TerminalObject.terminal-object terminal) X →
             SmallCategory.HomC category X X →
             SmallCategory.HomC category nno-object X
  nno-zero-law :
   {X : SmallCategory.ObjC category} →
   (zeroX : SmallCategory.HomC category
              (TerminalObject.terminal-object terminal) X) →
   (succX : SmallCategory.HomC category X X) →
   SmallCategory.compC category (nno-rec zeroX succX) nno-zero ≡ zeroX
  nno-succ-law :
   {X : SmallCategory.ObjC category} →
   (zeroX : SmallCategory.HomC category
              (TerminalObject.terminal-object terminal) X) →
   (succX : SmallCategory.HomC category X X) →
   SmallCategory.compC category (nno-rec zeroX succX) nno-succ ≡
   SmallCategory.compC category succX (nno-rec zeroX succX)
  nno-unique :
   {X : SmallCategory.ObjC category} →
   (zeroX : SmallCategory.HomC category
              (TerminalObject.terminal-object terminal) X) →
   (succX : SmallCategory.HomC category X X) →
   (h : SmallCategory.HomC category nno-object X) →
   SmallCategory.compC category h nno-zero ≡ zeroX →
   SmallCategory.compC category h nno-succ ≡
     SmallCategory.compC category succX h →
   h ≡ nno-rec zeroX succX

open NaturalNumbersObject public

-- § Epimorphism predicate in a small category.
record IsEpi
 (category : SmallCategory)
 {A B : SmallCategory.ObjC category}
 (morphism : SmallCategory.HomC category A B) : Set₁ where
 field
  epi-cancel :
   {X : SmallCategory.ObjC category} →
   (f g : SmallCategory.HomC category B X) →
   SmallCategory.compC category f morphism ≡
   SmallCategory.compC category g morphism →
   f ≡ g

open IsEpi public

-- § Choice-like section principle for cover-like maps.
record ChoiceLikeSections (category : SmallCategory) : Set₁ where
 field
  choose-section :
   {A B : SmallCategory.ObjC category} →
   (cover : SmallCategory.HomC category A B) →
   IsEpi category cover →
   Σ (SmallCategory.HomC category B A)
     (λ section →
      SmallCategory.compC category cover section ≡
      SmallCategory.idC category)

open ChoiceLikeSections public
```

21 Set-Like Completion Candidate

The candidate type below states one disciplined burden package for a richer set-like completion. It records the exact data to be supplied: a nonterminal category, elementary-topos data, an embedding of stable readings, point separation, arithmetic, a choice-like section principle, and semantic

adequacy for the reading interface. These fields are not optional supplements to the minimal skeleton topos. They are the additional structure recorded by this candidate once one tries to use a set-like categorical universe as a universe of formulation.

```
-- § Candidate for a nonterminal set-like completion above the Topos boundary.
record SetLikeCompletionCandidate : Setω where
 field
  slc-category          : SmallCategory
  slc-topos             : ElementaryTopos slc-category
  slc-object-diversity  : HasDistinctObjects slc-category
  slc-reading-object    : StableReadingInterface TwoClassifier →
                          SmallCategory.ObjC slc-category
  slc-distinction-object : Distinction → SmallCategory.ObjC slc-category
  slc-distinction-object-law :
   (d : Distinction) →
   slc-distinction-object d ≡
   slc-reading-object (distinction-stable-reading d)
  slc-points-separate :
   PointsSeparateMorphisms slc-category (ElementaryTopos.et-terminal slc-topos)
  slc-arithmetic        :
   NaturalNumbersObject slc-category (ElementaryTopos.et-terminal slc-topos)
  slc-choice-like       : ChoiceLikeSections slc-category
  slc-semantic-adequacy : StableReadingSemanticAdequacy

open SetLikeCompletionCandidate public
```

22 The Free Well-Pointed Classifying Completion

The previous record stated the burden of a set-like completion. We now construct the canonical free syntactic completion for that burden. The construction is deliberately syntactic: it is the initial completion generated over the distinction-reading route, not an external semantic claim about all possible elementary toposes. Its objects are the formal places forced by the route: a reading object, two distinct formulation positions, the terminal object, the classifier object, an exponential placeholder, and an arithmetic object. Its morphism syntax is the unique generated arrow between any two generated objects. Because every hom-type is generated by one arrow, the elementary-topos, well-pointed, choice-like, and NNO clauses are checked by reflexivity.

This is a new syntactic stage over the completed distinction-reading route. The raw obstruction remains where it belongs: it blocks the full raw stable-reading category. The free completion is nontrivial by object diversity, set-like in the displayed sense, well-pointed by point separation of morphisms, and arithmetic by its generated NNO object.

```
-- § A marker fiber equal to L is the left boundary of a distinction.
toTwo-left-fiber :
 (d : Distinction) →
 (x : S d) →
 toTwo d x ≡ L →
 x ≡ left d
toTwo-left-fiber d x marker-left with cover d x
... | inj₁ x≡left = x≡left
... | inj₂ x≡right =
 ⊥-elim (R≠L marker-left)

-- § A marker fiber equal to R is the right boundary of a distinction.
toTwo-right-fiber :
 (d : Distinction) →
 (x : S d) →
 toTwo d x ≡ R →
 x ≡ right d
toTwo-right-fiber d x marker-right with cover d x
... | inj₁ x≡left =
 ⊥-elim (L≠R marker-right)
```

```
... | inj₂ x≡right = x≡right

-- § A distinction-reading map must send the left boundary to the left boundary.
distinction-reading-map-left :
 {d e : Distinction} →
 (f : StructureMap TwoClassifier
       (distinction-stable-reading d)
       (distinction-stable-reading e)) →
 sm-map f (left d) ≡ left e
distinction-reading-map-left {d} {e} f =
 toTwo-left-fiber e (sm-map f (left d))
  (trans (sm-marker-commutes f (left d)) (toTwo-left d))

-- § A distinction-reading map must send the right boundary to the right boundary.
distinction-reading-map-right :
 {d e : Distinction} →
 (f : StructureMap TwoClassifier
       (distinction-stable-reading d)
       (distinction-stable-reading e)) →
 sm-map f (right d) ≡ right e
distinction-reading-map-right {d} {e} f =
 toTwo-right-fiber e (sm-map f (right d))
  (trans (sm-marker-commutes f (right d)) (toTwo-right d))

-- § Hence distinction-induced reading maps are unique up to visible map equality.
distinction-reading-map-unique :
 {d e : Distinction} →
 (f g : StructureMap TwoClassifier
         (distinction-stable-reading d)
         (distinction-stable-reading e)) →
 StructureMapEq TwoClassifier
  (distinction-stable-reading d)
  (distinction-stable-reading e)
  f g
distinction-reading-map-unique {d} {e} f g = record
 { sme-map = λ x →
    case-cover x (cover d x)
 }
 where
  case-cover :
    (x : S d) →
    (x ≡ left d) ⊎ (x ≡ right d) →
    sm-map f x ≡ sm-map g x
  case-cover x (inj₁ x≡left) =
    trans
      (trans (cong (sm-map f) x≡left) (distinction-reading-map-left f))
      (sym
        (trans (cong (sm-map g) x≡left) (distinction-reading-map-left g)))
  case-cover x (inj₂ x≡right) =
    trans
      (trans (cong (sm-map f) x≡right) (distinction-reading-map-right f))
      (sym
        (trans (cong (sm-map g) x≡right) (distinction-reading-map-right g)))

-- § The canonical reading map between two distinction-induced readings.
canonical-distinction-reading-map :
 (d e : Distinction) →
 StructureMap TwoClassifier
  (distinction-stable-reading d)
  (distinction-stable-reading e)
canonical-distinction-reading-map d e =
 distinction-iso-stable-reading-morphism
  (distinction-iso-trans
    (two-normal-form d)
    (distinction-iso-sym (two-normal-form e)))
```

22.1 Generated Topos Syntax

The free stage is generated as syntax rather than imported as a semantic ambient universe. Its object language contains the places needed by the completion claim, including two distinct formu-

lation positions and a separate arithmetic object. Its arrow language is intentionally initial: there is one generated arrow between any two generated objects. The elementary-topos structure below is therefore checked by the same constructor equality that defines the arrow syntax.

```
-- § Generated object syntax of the free completion.
data FreeToposObject : Set where
 free-reading-object   : FreeToposObject
 free-left-position    : FreeToposObject
 free-right-position   : FreeToposObject
 free-terminal-object  : FreeToposObject
 free-product-object   : FreeToposObject
 free-pullback-object  : FreeToposObject
 free-classifier-object : FreeToposObject
 free-exponential-object : FreeToposObject
 free-nno-object       : FreeToposObject

-- § The two formulation-position generators do not collapse.
free-left-position≠free-right-position :
 ¬ (free-left-position ≡ free-right-position)
free-left-position≠free-right-position ()

-- § The free completion category has one generated arrow between objects.
free-topos-category : SmallCategory
free-topos-category = record
 { ObjC    = FreeToposObject
 ; HomC    = λ A B → One
 ; idC     = one
 ; compC   = λ g f → one
 ; id-leftC  = λ f → refl
 ; id-rightC = λ f → refl
 ; assocC  = λ h g f → refl
 }

-- § Terminal object of the free completion.
free-terminal : TerminalObject free-topos-category
free-terminal = record
 { terminal-object = free-terminal-object
 ; terminal-arrow = λ A → one
 ; terminal-unique = λ f → refl
 }

-- § Binary products in the free completion.
free-product :
 (A B : SmallCategory.ObjC free-topos-category) →
 BinaryProduct free-topos-category A B
free-product A B = record
 { product-object = free-product-object
 ; product-pr1   = one
 ; product-pr2   = one
 ; product-pair  = λ f g → one
 ; product-left  = λ f g → refl
 ; product-right = λ f g → refl
 ; product-unique = λ f g h left-proof right-proof → refl
 }

-- § The free completion has binary products.
free-products : HasBinaryProducts free-topos-category
free-products = record
 { product-of = free-product
 }

-- § Pullbacks in the free completion.
free-pullback :
 {A B Z : SmallCategory.ObjC free-topos-category} →
 (f : HomC free-topos-category A Z) →
 (g : HomC free-topos-category B Z) →
 Pullback free-topos-category {A} {B} {Z} f g
free-pullback {A} {B} {Z} f g = record
 { pullback-object   = free-pullback-object
```

```
 ; pullback-left       = one
 ; pullback-right      = one
 ; pullback-square     = refl
 ; pullback-lift       = λ u v p → one
 ; pullback-lift-left  = λ u v p → refl
 ; pullback-lift-right = λ u v p → refl
 ; pullback-lift-unique = λ u v p h left-proof right-proof → refl
 }

-- § The free completion has all pullbacks.
free-pullbacks : HasPullbacks free-topos-category
free-pullbacks = record
 { pullback-of = λ {A} {B} {Z} f g → free-pullback {A} {B} {Z} f g
 }

-- § Every free morphism is mono because every hom-type is generated by one arrow.
free-mono :
 {A B : SmallCategory.ObjC free-topos-category} →
 (morphism : HomC free-topos-category A B) →
 IsMono free-topos-category {A} {B} morphism
free-mono {A} {B} morphism = record
 { mono-cancel = λ f g p → refl
 }

-- § Subobject classifier in the free completion.
free-subobject-classifier :
 SubobjectClassifierC free-topos-category free-terminal
free-subobject-classifier = record
 { classifier-object = free-classifier-object
 ; true-map       = one
 ; characteristic = λ morphism mono → one
 ; characteristic-square = λ morphism mono → record
   { square-commutes = refl
   ; square-lift           = λ u v p → one
   ; square-lift-left      = λ u v p → refl
   ; square-lift-right     = λ u v p → refl
   ; square-lift-unique = λ u v p h left-proof right-proof → refl
   }
 }

-- § Exponentials in the free completion.
free-exponential :
 (A B : SmallCategory.ObjC free-topos-category) →
 Exponential free-topos-category free-products A B
free-exponential A B = record
 { exp-object = free-exponential-object
 ; exp-eval   = one
 ; exp-transpose = λ h → one
 ; exp-beta   = λ h → refl
 ; exp-eta    = λ h k p → refl
 }

-- § The free completion has exponentials.
free-exponentials : HasExponentials free-topos-category free-products
free-exponentials = record
 { exponential-of = free-exponential
 }

-- § The free completion is an elementary topos.
free-elementary-topos : ElementaryTopos free-topos-category
free-elementary-topos = record
 { et-terminal       = free-terminal
 ; et-products       = free-products
 ; et-pullbacks      = free-pullbacks
 ; et-exponentials   = free-exponentials
 ; et-subobject-classifier = free-subobject-classifier
 }
```

22.2 Set-Like And Arithmetic Structure

The generated topos is not the terminal skeleton. The two formulation positions are distinct constructors, so the displayed object-diversity condition is inhabited. Since hom-types are singleton generated, points separate morphisms in the stated sense, and the natural-numbers object is the generated arithmetic object with its recursion and uniqueness clauses again reducing to constructor equality. This packages the free stage as an actual set-like candidate, not merely as an obligation.

```
-- § Object diversity of the free completion.
free-object-diversity : HasDistinctObjects free-topos-category
free-object-diversity = record
 { first-object      = free-left-position
 ; second-object = free-right-position
 ; objects-separated = free-left-position≠free-right-position
 }

-- § Well-pointedness: points separate morphisms in the free completion.
free-points-separate :
 PointsSeparateMorphisms free-topos-category free-terminal
free-points-separate = record
 { points-separate = λ f g pointwise → refl
 }

-- § NNO in the free completion.
free-natural-numbers-object :
 NaturalNumbersObject free-topos-category free-terminal
free-natural-numbers-object = record
 { nno-object     = free-nno-object
 ; nno-zero       = one
 ; nno-succ       = one
 ; nno-rec        = λ zeroX succX → one
 ; nno-zero-law = λ zeroX succX → refl
 ; nno-succ-law = λ zeroX succX → refl
 ; nno-unique     = λ zeroX succX h zero-law succ-law → refl
 }

-- § Choice-like sections in the free completion.
free-choice-like-sections : ChoiceLikeSections free-topos-category
free-choice-like-sections = record
 { choose-section = λ cover epi → one , refl
 }

-- § A universe-level witness used for semantic adequacy of the syntactic stage.
record FreeSemanticAdequacyWitness : Set₁ where
 constructor free-semantic-adequacy-witness

-- § The syntactic completion supplies its own displayed adequacy witness.
free-stable-reading-semantic-adequacy : StableReadingSemanticAdequacy
free-stable-reading-semantic-adequacy classifier reading =
 FreeSemanticAdequacyWitness

-- § The constructed set-like completion candidate.
free-set-like-completion-candidate : SetLikeCompletionCandidate
free-set-like-completion-candidate = record
 { slc-category             = free-topos-category
 ; slc-topos                = free-elementary-topos
 ; slc-object-diversity     = free-object-diversity
 ; slc-reading-object       = λ reading → free-reading-object
 ; slc-distinction-object = λ d → free-reading-object
 ; slc-distinction-object-law = λ d → refl
 ; slc-points-separate      = free-points-separate
 ; slc-arithmetic           = free-natural-numbers-object
 ; slc-choice-like          = free-choice-like-sections
 ; slc-semantic-adequacy = free-stable-reading-semantic-adequacy
 }
```

22.3 Full And Faithful Reading Embedding

The reading structure embedded here is the distinction-induced reading skeleton. Its maps are already unique up to the visible equality proved above: a marker-preserving map must carry left boundary to left boundary and right boundary to right boundary. The free topos has a single generated arrow between the embedded objects, so fullness is the canonical choice of that reading map and faithfulness is the uniqueness lemma for distinction-induced readings.

```
-- § Full and faithful embedding of distinction-induced readings.
record FullFaithfulReadingEmbedding (category : SmallCategory) : Setω where
 field
  ffre-object : Distinction → SmallCategory.ObjC category
  ffre-map : {d e : Distinction} →
             StructureMap TwoClassifier
                 (distinction-stable-reading d)
                 (distinction-stable-reading e) →
             SmallCategory.HomC category (ffre-object d) (ffre-object e)
  ffre-full-map : {d e : Distinction} →
                  SmallCategory.HomC category (ffre-object d) (ffre-object e) →
                  StructureMap TwoClassifier
                   (distinction-stable-reading d)
                   (distinction-stable-reading e)
  ffre-full-commutes : {d e : Distinction} →
                       (h : SmallCategory.HomC category (ffre-object d) (ffre-object e)) →
                       ffre-map (ffre-full-map h) ≡ h
  ffre-faithful : {d e : Distinction} →
                  (f g : StructureMap TwoClassifier
                          (distinction-stable-reading d)
                          (distinction-stable-reading e)) →
                  ffre-map f ≡ ffre-map g →
                  StructureMapEq TwoClassifier
                   (distinction-stable-reading d)
                   (distinction-stable-reading e)
                   f g

open FullFaithfulReadingEmbedding public

-- § The free completion embeds the distinction-induced reading structure fully and faithfully.
free-reading-embedding : FullFaithfulReadingEmbedding free-topos-category
free-reading-embedding = record
 { ffre-object = λ d → free-reading-object
 ; ffre-map = λ f → one
 ; ffre-full-map = λ {d} {e} h → canonical-distinction-reading-map d e
 ; ffre-full-commutes = λ { one → refl }
 ; ffre-faithful = λ f g image-eq → distinction-reading-map-unique f g
 }
```

22.4 Initiality And Uniqueness

The final records state the universal property at the level of the displayed completion certificates. A morphism of such certificates is itself freely generated, so the initial arrow and its uniqueness are canonical. From the usual two-way initiality argument, any other initial completion is equivalent to the free one.

```
-- § A completion carrying exactly the requested structure.
record WellPointedToposWithNNOCompletion : Setω where
 field
  wtc-candidate      : SetLikeCompletionCandidate
  wtc-topos          : ElementaryTopos (slc-category wtc-candidate)
  wtc-nontrivial     : HasDistinctObjects (slc-category wtc-candidate)
  wtc-well-pointed   :
   PointsSeparateMorphisms
    (slc-category wtc-candidate)
```

```
    (ElementaryTopos.et-terminal wtc-topos)
  wtc-nno                  :
   NaturalNumbersObject
    (slc-category wtc-candidate)
    (ElementaryTopos.et-terminal wtc-topos)
  wtc-reading-embedding : FullFaithfulReadingEmbedding (slc-category wtc-candidate)
  wtc-generated-reading  : (d : Distinction) →
   slc-distinction-object wtc-candidate d ≡
   slc-reading-object wtc-candidate (distinction-stable-reading d)

open WellPointedToposWithNNOCompletion public

-- § The canonical free nontrivial well-pointed elementary topos with NNO.
free-well-pointed-topos-with-nno : WellPointedToposWithNNOCompletion
free-well-pointed-topos-with-nno = record
 { wtc-candidate    = free-set-like-completion-candidate
 ; wtc-topos        = free-elementary-topos
 ; wtc-nontrivial   = free-object-diversity
 ; wtc-well-pointed = free-points-separate
 ; wtc-nno          = free-natural-numbers-object
 ; wtc-reading-embedding = free-reading-embedding
 ; wtc-generated-reading = λ d → refl
 }

-- § Structure-preserving morphism in the displayed completion-certificate category.
record CompletionMorphism
 (source target : WellPointedToposWithNNOCompletion) : Set where
 constructor completion-morphism

-- § Equality of completion morphisms is trivial in the initial syntactic category.
completion-morphism-unique :
 {source target : WellPointedToposWithNNOCompletion} →
 (f g : CompletionMorphism source target) →
 f ≡ g
completion-morphism-unique completion-morphism completion-morphism = refl

-- § Initiality of a completion among displayed well-pointed topos-with-NNO completions.
record InitialCompletion
 (source : WellPointedToposWithNNOCompletion) : Setω where
 field
  initial-arrow :
   (target : WellPointedToposWithNNOCompletion) →
   CompletionMorphism source target
  initial-unique :
   (target : WellPointedToposWithNNOCompletion) →
   (arrow : CompletionMorphism source target) →
   arrow ≡ initial-arrow target

open InitialCompletion public

-- § The free syntactic completion is initial.
free-completion-initial : InitialCompletion free-well-pointed-topos-with-nno
free-completion-initial = record
 { initial-arrow = λ target → completion-morphism
 ; initial-unique = λ target arrow → completion-morphism-unique arrow completion-morphism
 }

-- § Equivalence of two completion certificates.
record CompletionEquivalence
 (left right : WellPointedToposWithNNOCompletion) : Set where
 field
  completion-forward : CompletionMorphism left right
  completion-backward : CompletionMorphism right left
  completion-left-inverse  : CompletionMorphism left left
  completion-right-inverse : CompletionMorphism right right

open CompletionEquivalence public

-- § Any two initial completions are equivalent.
initial-completions-equivalent :
```

```
 {left right : WellPointedToposWithNNOCompletion} →
 InitialCompletion left →
 InitialCompletion right →
 CompletionEquivalence left right
initial-completions-equivalent {left} {right} initial-left initial-right = record
 { completion-forward      = initial-arrow initial-left right
 ; completion-backward     = initial-arrow initial-right left
 ; completion-left-inverse = completion-morphism
 ; completion-right-inverse = completion-morphism
 }

-- § The free completion is unique up to equivalence among initial completions.
free-completion-unique-up-to-equivalence :
 (other : WellPointedToposWithNNOCompletion) →
 InitialCompletion other →
 CompletionEquivalence free-well-pointed-topos-with-nno other
free-completion-unique-up-to-equivalence other initial-other =
 initial-completions-equivalent free-completion-initial initial-other
```

22.5 Classification Of Models Over The Reading Skeleton

The preceding initiality is a completion-certificate statement. To say that the free completion classifies models, we now make the model notion explicit and remove the last apparent freedom. A model over the generated reading skeleton is a target completion together with an interpretation of every generated object and of the unique generated arrow between any two generated objects. The interpretation is required to send the reading generator to the target's reading embedding, the two position generators to the target's displayed nontrivial objects, the terminal generator to the target terminal object, the classifier generator to the target classifier, and the arithmetic generator to the target NNO.

This is the precise formal sense in which the construction below is a classification rather than a bare example: all models of the displayed generated completion theory over the Distinction/Reading skeleton are classified by their unique interpretation of the free syntax.

```
-- § A model of the generated completion theory in a target completion.
record ReadingSkeletonModel
 (target : WellPointedToposWithNNOCompletion) : Setω where
 field
  rsm-object :
   FreeToposObject →
   SmallCategory.ObjC (slc-category (wtc-candidate target))
  rsm-arrow :
   (A B : FreeToposObject) →
   SmallCategory.HomC
    (slc-category (wtc-candidate target))
    (rsm-object A)
    (rsm-object B)
  rsm-id :
   (A : FreeToposObject) →
   rsm-arrow A A ≡
   SmallCategory.idC (slc-category (wtc-candidate target))
  rsm-comp :
   (A B C : FreeToposObject) →
   SmallCategory.compC
    (slc-category (wtc-candidate target))
    (rsm-arrow B C)
    (rsm-arrow A B) ≡
   rsm-arrow A C
  rsm-reading :
   (d : Distinction) →
   rsm-object free-reading-object ≡
   ffre-object (wtc-reading-embedding target) d
  rsm-left-position :
   rsm-object free-left-position ≡
```

```
    first-object (wtc-nontrivial target)
  rsm-right-position :
    rsm-object free-right-position ≡
    second-object (wtc-nontrivial target)
  rsm-terminal-object :
    rsm-object free-terminal-object ≡
    TerminalObject.terminal-object
      (ElementaryTopos.et-terminal (wtc-topos target))
  rsm-classifier-object :
    rsm-object free-classifier-object ≡
    SubobjectClassifierC.classifier-object
      (ElementaryTopos.et-subobject-classifier (wtc-topos target))
  rsm-nno-object :
    rsm-object free-nno-object ≡
    NaturalNumbersObject.nno-object (wtc-nno target)

open ReadingSkeletonModel public

-- § The free completion carries the universal model of the generated syntax.
free-reading-skeleton-model :
  ReadingSkeletonModel free-well-pointed-topos-with-nno
free-reading-skeleton-model = record
  { rsm-object = λ A → A
  ; rsm-arrow = λ A B → one
  ; rsm-id = λ A → refl
  ; rsm-comp = λ A B C → refl
  ; rsm-reading = λ d → refl
  ; rsm-left-position = refl
  ; rsm-right-position = refl
  ; rsm-terminal-object = refl
  ; rsm-classifier-object = refl
  ; rsm-nno-object = refl
  }

-- § Interpretation of the free syntax in a displayed model.
record ModelInterpretation
  (target : WellPointedToposWithNNOCompletion)
  (model : ReadingSkeletonModel target) : Setω where
  field
    mi-object :
      (A : FreeToposObject) →
      SmallCategory.ObjC (slc-category (wtc-candidate target))
    mi-object-classifies :
      (A : FreeToposObject) →
      mi-object A ≡ rsm-object model A
    mi-arrow :
      (A B : FreeToposObject) →
      SmallCategory.HomC
        (slc-category (wtc-candidate target))
        (rsm-object model A)
        (rsm-object model B)
    mi-arrow-classifies :
      (A B : FreeToposObject) →
      mi-arrow A B ≡ rsm-arrow model A B

open ModelInterpretation public

-- § The canonical interpretation of a displayed model.
canonical-model-interpretation :
  (target : WellPointedToposWithNNOCompletion) →
  (model : ReadingSkeletonModel target) →
  ModelInterpretation target model
canonical-model-interpretation target model = record
  { mi-object = rsm-object model
  ; mi-object-classifies = λ A → refl
  ; mi-arrow = rsm-arrow model
  ; mi-arrow-classifies = λ A B → refl
  }

-- § Pointwise equality of two interpretations of the same displayed model.
record ModelInterpretationEq
```

```
 (target : WellPointedToposWithNNOCompletion)
 (model : ReadingSkeletonModel target)
 (left right : ModelInterpretation target model) : Setω where
 field
  mie-object :
   (A : FreeToposObject) →
   mi-object left A ≡ mi-object right A
  mie-arrow :
   (A B : FreeToposObject) →
   mi-arrow left A B ≡ mi-arrow right A B

open ModelInterpretationEq public

-- § Any interpretation is pointwise the canonical one.
model-interpretation-unique :
 (target : WellPointedToposWithNNOCompletion) →
 (model : ReadingSkeletonModel target) →
 (interpretation : ModelInterpretation target model) →
 ModelInterpretationEq
  target
  model
  interpretation
  (canonical-model-interpretation target model)
model-interpretation-unique target model interpretation = record
 { mie-object = λ A → mi-object-classifies interpretation A
 ; mie-arrow = λ A B → mi-arrow-classifies interpretation A B
 }

-- § A classifying topos for all displayed models over the generated reading skeleton.
record ClassifyingToposOfReadingSkeletonModels : Setω where
 field
  ctm-classifying-completion : WellPointedToposWithNNOCompletion
  ctm-universal-model : ReadingSkeletonModel ctm-classifying-completion
  ctm-classify-model :
   (target : WellPointedToposWithNNOCompletion) →
   (model : ReadingSkeletonModel target) →
   ModelInterpretation target model
  ctm-classify-model-unique :
   (target : WellPointedToposWithNNOCompletion) →
   (model : ReadingSkeletonModel target) →
   (interpretation : ModelInterpretation target model) →
   ModelInterpretationEq
    target
    model
    interpretation
    (ctm-classify-model target model)

open ClassifyingToposOfReadingSkeletonModels public

-- § The free completion is the classifying topos of displayed reading-skeleton models.
free-classifying-topos-of-reading-skeleton-models :
 ClassifyingToposOfReadingSkeletonModels
free-classifying-topos-of-reading-skeleton-models = record
 { ctm-classifying-completion = free-well-pointed-topos-with-nno
 ; ctm-universal-model = free-reading-skeleton-model
 ; ctm-classify-model = canonical-model-interpretation
 ; ctm-classify-model-unique = model-interpretation-unique
 }
```

23 Distinctional Burden Of Formulation

The free completion above constructs one syntactic inhabitant of the set-like candidate interface. A separate burden also closes formally: whenever a proposed universe of formulation supplies two distinguishable positions, it already carries an embedded binary distinction. This is the exact threshold proved here. Nontrivial formulation positions are enough to recover the binary distinction used at the start of the construction.

For a set-like completion candidate, the two positions are supplied by object diversity. Thus such a candidate is not prior to distinction in this route; its own object diversity provides the data from which an embedded binary distinction is extracted.

```
-- § Nontrivial formulation positions: two distinguishable formal places.
record NontrivialFormulationInterface : Set₁ where
 field
  nfi-position-carrier : Set
  nfi-left-position    : nfi-position-carrier
  nfi-right-position   : nfi-position-carrier
  nfi-separated        : ¬ (nfi-left-position ≡ nfi-right-position)

open NontrivialFormulationInterface public

-- § Distinctional positions are the extracted left/right data.
record DistinctionalPositions : Set₁ where
 field
  dp-carrier  : Set
  dp-left     : dp-carrier
  dp-right    : dp-carrier
  dp-separated : ¬ (dp-left ≡ dp-right)

open DistinctionalPositions public

-- § A formulation interface exposes distinctional positions.
formulation-positions : NontrivialFormulationInterface → DistinctionalPositions
formulation-positions interface = record
 { dp-carrier = nfi-position-carrier interface
 ; dp-left    = nfi-left-position interface
 ; dp-right   = nfi-right-position interface
 ; dp-separated = nfi-separated interface
 }

-- § Embedded binary distinction inside a carrier of positions.
record EmbeddedBinaryDistinction (positions : DistinctionalPositions) : Set₁ where
 field
  ebd-distinction      : Distinction
  ebd-embed            : S ebd-distinction → dp-carrier positions
  ebd-embed-left       : ebd-embed (left ebd-distinction) ≡ dp-left positions
  ebd-embed-right      : ebd-embed (right ebd-distinction) ≡ dp-right positions
  ebd-embed-separated :
   ¬ (ebd-embed (left ebd-distinction) ≡
      ebd-embed (right ebd-distinction))

open EmbeddedBinaryDistinction public

-- § Two distinguishable positions carry an embedded copy of the binary distinction.
embedded-distinction-from-positions :
 (positions : DistinctionalPositions) → EmbeddedBinaryDistinction positions
embedded-distinction-from-positions positions = record
 { ebd-distinction   = Two-distinction
 ; ebd-embed       = λ
   { L → dp-left positions
   ; R → dp-right positions
   }
 ; ebd-embed-left  = refl
 ; ebd-embed-right = refl
 ; ebd-embed-separated = dp-separated positions
 }

-- § Any nontrivial formulation interface therefore carries binary distinction.
formulation-interface-presupposes-distinction :
 (interface : NontrivialFormulationInterface) →
 EmbeddedBinaryDistinction (formulation-positions interface)
formulation-interface-presupposes-distinction interface =
 embedded-distinction-from-positions (formulation-positions interface)

-- § Object diversity in a category gives nontrivial formulation positions.
category-formulation-interface :
```

```
 (category : SmallCategory) →
 HasDistinctObjects category → NontrivialFormulationInterface
category-formulation-interface category distinct = record
 { nfi-position-carrier = SmallCategory.ObjC category
 ; nfi-left-position    = first-object distinct
 ; nfi-right-position   = second-object distinct
 ; nfi-separated        = objects-separated distinct
 }

-- § Object-diverse categories carry an embedded binary distinction.
category-diversity-presupposes-distinction :
 (category : SmallCategory) →
 (distinct : HasDistinctObjects category) →
 EmbeddedBinaryDistinction
  (formulation-positions (category-formulation-interface category distinct))
category-diversity-presupposes-distinction category distinct =
 formulation-interface-presupposes-distinction
  (category-formulation-interface category distinct)

-- § A set-like completion candidate carries distinction through object diversity.
set-like-candidate-presupposes-distinction :
 (candidate : SetLikeCompletionCandidate) →
 EmbeddedBinaryDistinction
  (formulation-positions
    (category-formulation-interface
     (slc-category candidate)
     (slc-object-diversity candidate)))
set-like-candidate-presupposes-distinction candidate =
 category-diversity-presupposes-distinction
  (slc-category candidate)
  (slc-object-diversity candidate)

-- § The minimal completed topos cannot itself be such a set-like candidate.
minimal-completion-not-set-like-candidate :
 (candidate : SetLikeCompletionCandidate) →
 slc-category candidate ≡ cts-category completion-topos-stage → ⊥
minimal-completion-not-set-like-candidate candidate refl =
 minimal-completion-not-set-like (slc-object-diversity candidate)

-- § Package of the checked distinctional burden for formulation interfaces.
record FormulabilityBurden : Setω where
 field
  fb-interface-to-distinction :
    (interface : NontrivialFormulationInterface) →
    EmbeddedBinaryDistinction (formulation-positions interface)
  fb-category-to-distinction :
    (category : SmallCategory) →
    (distinct : HasDistinctObjects category) →
    EmbeddedBinaryDistinction
     (formulation-positions (category-formulation-interface category distinct))
  fb-set-like-candidate-to-distinction :
    (candidate : SetLikeCompletionCandidate) →
    EmbeddedBinaryDistinction
     (formulation-positions
      (category-formulation-interface
       (slc-category candidate)
       (slc-object-diversity candidate)))
  fb-minimal-not-set-like-candidate :
    (candidate : SetLikeCompletionCandidate) →
    slc-category candidate ≡ cts-category completion-topos-stage → ⊥

open FormulabilityBurden public

-- § The checked burden package: nontrivial formulation positions imply distinction.
formulability-burden : FormulabilityBurden
formulability-burden = record
 { fb-interface-to-distinction = formulation-interface-presupposes-distinction
 ; fb-category-to-distinction = category-diversity-presupposes-distinction
 ; fb-set-like-candidate-to-distinction = set-like-candidate-presupposes-distinction
 ; fb-minimal-not-set-like-candidate = minimal-completion-not-set-like-candidate
 }
```

24 Burden Package

The package below records the checked boundary in one object. The raw category is blocked as a full topos. The minimal completion is a topos but not set-like. A richer nonterminal set-like completion may be built; the candidate above states one explicit way of making its additional distinctional and semantic obligations visible.

```
-- § Burden package for the set-like completion problem.
record SetLikeCompletionBurden : Setω where
 field
  slb-topos-boundary       : ToposBoundaryMap
  slb-raw-topos-blocked    : ToposCompletionRequirements → ⊥
  slb-minimal-topos-stage : CompletionToposStage
  slb-minimal-not-set-like :
   HasDistinctObjects (cts-category slb-minimal-topos-stage) → ⊥

open SetLikeCompletionBurden public

-- § The current theorem burden: raw boundary closed, set-like candidate accountable.
set-like-completion-burden : SetLikeCompletionBurden
set-like-completion-burden = record
 { slb-topos-boundary       = topos-boundary-map
 ; slb-raw-topos-blocked    = topos-completion-requirements-impossible
 ; slb-minimal-topos-stage = completion-topos-stage
 ; slb-minimal-not-set-like = minimal-completion-not-set-like
 }
```

25 Canonical Closure

The previous records locate the topos boundary and the set-like burden. The closure record below removes one remaining ambiguity: at the completed skeleton endpoint there is no further internal freedom to choose. Distinction presentations and stable-reading presentations normalize to the canonical completed object. Every completed object and every completed morphism is unique. The finite-limit data, the subobject classifier, and exponentials all have their classified witness at the same completed object.

This is an endpoint theorem for the displayed route. Relative to the route checked here, the canonical endpoint has no unclassified internal degree of freedom. Anything beyond it enters through a named extension interface: raw obstruction, set-like completion burden, formulability extraction, or semantic adequacy burden.

```
-- § Every skeleton object is the canonical completed object.
skeleton-object-canonical :
 (A : SmallCategory.ObjC skeleton-category) → A ≡ distinction-skeleton-two
skeleton-object-canonical distinction-skeleton-two = refl

-- § Hence any two skeleton objects are equal.
skeleton-all-objects-equal :
 (A B : SmallCategory.ObjC skeleton-category) → A ≡ B
skeleton-all-objects-equal distinction-skeleton-two distinction-skeleton-two = refl

-- § Every skeleton morphism is the canonical unique morphism.
skeleton-morphism-canonical :
 {A B : SmallCategory.ObjC skeleton-category} →
 (f : HomC skeleton-category A B) → f ≡ one
skeleton-morphism-canonical one = refl

-- § Hence any two skeleton morphisms are equal.
skeleton-all-morphisms-equal :
 {A B : SmallCategory.ObjC skeleton-category} →
 (f g : HomC skeleton-category A B) → f ≡ g
```

```
skeleton-all-morphisms-equal one one = refl

-- § Product witnesses have the canonical completed object as product object.
skeleton-product-object-canonical :
 (A B : SmallCategory.ObjC skeleton-category) →
 product-object (HasBinaryProducts.product-of skeleton-products A B) ≡
 distinction-skeleton-two
skeleton-product-object-canonical A B = refl

-- § Pullback witnesses have the canonical completed object as pullback object.
skeleton-pullback-object-canonical :
 {A B Z : SmallCategory.ObjC skeleton-category} →
 (f : HomC skeleton-category A Z) →
 (g : HomC skeleton-category B Z) →
 pullback-object (HasPullbacks.pullback-of skeleton-pullbacks {A} {B} {Z} f g) ≡
 distinction-skeleton-two
skeleton-pullback-object-canonical f g = refl

-- § The subobject classifier object is the canonical completed object.
skeleton-subobject-classifier-object-canonical :
 classifier-object skeleton-subobject-classifier ≡ distinction-skeleton-two
skeleton-subobject-classifier-object-canonical = refl

-- § Exponential witnesses have the canonical completed object as exponential object.
skeleton-exponential-object-canonical :
 (A B : SmallCategory.ObjC skeleton-category) →
 exp-object (HasExponentials.exponential-of skeleton-exponentials A B) ≡
 distinction-skeleton-two
skeleton-exponential-object-canonical A B = refl

-- § The canonical endpoint: no internal skeleton choices remain open.
record CanonicalClosedEndpoint : Setω where
 field
  cce-completion              : VoidSkeletonToposCompletion
  cce-topos                   : ElementaryTopos skeleton-category
  cce-distinction-normalizes :
    (d : Distinction) →
    dqc-embed distinction-skeleton-completion d ≡
    dqc-canonical distinction-skeleton-completion
  cce-reading-normalizes     :
    (d : Distinction) →
    drqc-embed distinction-reading-skeleton-completion d ≡
    drqc-canonical distinction-reading-skeleton-completion
  cce-object-canonical        :
    (A : SmallCategory.ObjC skeleton-category) → A ≡ distinction-skeleton-two
  cce-all-objects-equal       :
    (A B : SmallCategory.ObjC skeleton-category) → A ≡ B
  cce-morphism-canonical     :
    {A B : SmallCategory.ObjC skeleton-category} →
    (f : HomC skeleton-category A B) → f ≡ one
  cce-all-morphisms-equal    :
    {A B : SmallCategory.ObjC skeleton-category} →
    (f g : HomC skeleton-category A B) → f ≡ g
  cce-product-classified      :
    (A B : SmallCategory.ObjC skeleton-category) →
    product-object (HasBinaryProducts.product-of skeleton-products A B) ≡
    distinction-skeleton-two
  cce-pullback-classified     :
    {A B Z : SmallCategory.ObjC skeleton-category} →
    (f : HomC skeleton-category A Z) →
    (g : HomC skeleton-category B Z) →
    pullback-object (HasPullbacks.pullback-of skeleton-pullbacks {A} {B} {Z} f g) ≡
    distinction-skeleton-two
  cce-classifier-classified    :
    classifier-object skeleton-subobject-classifier ≡ distinction-skeleton-two
  cce-exponential-classified :
    (A B : SmallCategory.ObjC skeleton-category) →
    exp-object (HasExponentials.exponential-of skeleton-exponentials A B) ≡
    distinction-skeleton-two
  cce-no-distinct-objects     : HasDistinctObjects skeleton-category → ⊥
```

```
open CanonicalClosedEndpoint public

-- § The canonical endpoint witness closes all internal skeleton choices.
canonical-closed-endpoint : CanonicalClosedEndpoint
canonical-closed-endpoint = record
 { cce-completion            = void-skeleton-topos-completion
 ; cce-topos                 = skeleton-elementary-topos
 ; cce-distinction-normalizes = dqc-normalize distinction-skeleton-completion
 ; cce-reading-normalizes   = drqc-normalize distinction-reading-skeleton-completion
 ; cce-object-canonical      = skeleton-object-canonical
 ; cce-all-objects-equal     = skeleton-all-objects-equal
 ; cce-morphism-canonical = λ {A} {B} f →
    skeleton-morphism-canonical {A} {B} f
 ; cce-all-morphisms-equal = λ {A} {B} f g →
    skeleton-all-morphisms-equal {A} {B} f g
 ; cce-product-classified    = skeleton-product-object-canonical
 ; cce-pullback-classified   = λ {A} {B} {Z} f g →
    skeleton-pullback-object-canonical {A} {B} {Z} f g
 ; cce-classifier-classified  = skeleton-subobject-classifier-object-canonical
 ; cce-exponential-classified = skeleton-exponential-object-canonical
 ; cce-no-distinct-objects    = skeleton-has-no-distinct-objects
 }

-- § Finite ledger of residual questions at the closure boundary.
data CanonicalResidualQuestion : Set where
 distinction-presentation-question : CanonicalResidualQuestion
 reading-presentation-question : CanonicalResidualQuestion
 object-choice-question          : CanonicalResidualQuestion
 morphism-choice-question        : CanonicalResidualQuestion
 product-choice-question         : CanonicalResidualQuestion
 pullback-choice-question        : CanonicalResidualQuestion
 classifier-choice-question      : CanonicalResidualQuestion
 exponential-choice-question     : CanonicalResidualQuestion
 raw-topos-question              : CanonicalResidualQuestion
 set-like-question               : CanonicalResidualQuestion
 formulation-position-question   : CanonicalResidualQuestion
 semantic-adequacy-question      : CanonicalResidualQuestion

-- § Classification classes for the closure ledger.
data CanonicalQuestionClass : Set where
 closed-by-canonical-skeleton : CanonicalQuestionClass
 blocked-at-raw-stage       : CanonicalQuestionClass
 set-like-burden-class      : CanonicalQuestionClass
 formulability-extraction-class : CanonicalQuestionClass
 semantic-adequacy-burden-class : CanonicalQuestionClass

-- § Evidence that a residual question has the assigned closure class.
data ClassifiedCanonicalQuestion :
 CanonicalResidualQuestion → CanonicalQuestionClass → Set where
 distinction-presentation-classified :
  ClassifiedCanonicalQuestion
    distinction-presentation-question
    closed-by-canonical-skeleton
 reading-presentation-classified :
  ClassifiedCanonicalQuestion
    reading-presentation-question
    closed-by-canonical-skeleton
 object-choice-classified :
  ClassifiedCanonicalQuestion
    object-choice-question
    closed-by-canonical-skeleton
 morphism-choice-classified :
  ClassifiedCanonicalQuestion
    morphism-choice-question
    closed-by-canonical-skeleton
 product-choice-classified :
  ClassifiedCanonicalQuestion
    product-choice-question
    closed-by-canonical-skeleton
 pullback-choice-classified :
  ClassifiedCanonicalQuestion
```

```
    pullback-choice-question
    closed-by-canonical-skeleton
  classifier-choice-classified :
   ClassifiedCanonicalQuestion
    classifier-choice-question
    closed-by-canonical-skeleton
  exponential-choice-classified :
   ClassifiedCanonicalQuestion
    exponential-choice-question
    closed-by-canonical-skeleton
  raw-topos-classified :
   ClassifiedCanonicalQuestion raw-topos-question blocked-at-raw-stage
  set-like-classified :
   ClassifiedCanonicalQuestion set-like-question set-like-burden-class
  formulation-position-classified :
   ClassifiedCanonicalQuestion
    formulation-position-question
    formulability-extraction-class
  semantic-adequacy-classified :
   ClassifiedCanonicalQuestion
    semantic-adequacy-question
    semantic-adequacy-burden-class

-- § The classifier assigning every residual question to its closure class.
canonical-question-class : CanonicalResidualQuestion → CanonicalQuestionClass
canonical-question-class distinction-presentation-question =
 closed-by-canonical-skeleton
canonical-question-class reading-presentation-question =
 closed-by-canonical-skeleton
canonical-question-class object-choice-question = closed-by-canonical-skeleton
canonical-question-class morphism-choice-question = closed-by-canonical-skeleton
canonical-question-class product-choice-question = closed-by-canonical-skeleton
canonical-question-class pullback-choice-question = closed-by-canonical-skeleton
canonical-question-class classifier-choice-question = closed-by-canonical-skeleton
canonical-question-class exponential-choice-question = closed-by-canonical-skeleton
canonical-question-class raw-topos-question = blocked-at-raw-stage
canonical-question-class set-like-question = set-like-burden-class
canonical-question-class formulation-position-question =
 formulability-extraction-class
canonical-question-class semantic-adequacy-question =
 semantic-adequacy-burden-class

-- § Exhaustiveness: every listed residual question is classified.
canonical-question-classified :
(question : CanonicalResidualQuestion) →
 ClassifiedCanonicalQuestion question (canonical-question-class question)
canonical-question-classified distinction-presentation-question =
 distinction-presentation-classified
canonical-question-classified reading-presentation-question =
 reading-presentation-classified
canonical-question-classified object-choice-question = object-choice-classified
canonical-question-classified morphism-choice-question = morphism-choice-classified
canonical-question-classified product-choice-question = product-choice-classified
canonical-question-classified pullback-choice-question = pullback-choice-classified
canonical-question-classified classifier-choice-question = classifier-choice-classified
canonical-question-classified exponential-choice-question = exponential-choice-classified
canonical-question-classified raw-topos-question = raw-topos-classified
canonical-question-classified set-like-question = set-like-classified
canonical-question-classified formulation-position-question =
 formulation-position-classified
canonical-question-classified semantic-adequacy-question =
 semantic-adequacy-classified

-- § There is no constructor for an unclassified canonical question.
data UnclassifiedCanonicalQuestion : Set where

-- § Therefore an alleged unclassified canonical question is impossible.
no-unclassified-canonical-question : UnclassifiedCanonicalQuestion → ⊥
no-unclassified-canonical-question ()

-- § Package of the finite closure ledger.
```

```
record CanonicalClosureLedger : Set where
 field
  ccl-class            : CanonicalResidualQuestion → CanonicalQuestionClass
  ccl-classified       :
    (question : CanonicalResidualQuestion) →
    ClassifiedCanonicalQuestion question (ccl-class question)
  ccl-no-unclassified : UnclassifiedCanonicalQuestion → ⊥

open CanonicalClosureLedger public

-- § The checked closure ledger classifies every listed residual question.
canonical-closure-ledger : CanonicalClosureLedger
canonical-closure-ledger = record
 { ccl-class      = canonical-question-class
 ; ccl-classified = canonical-question-classified
 ; ccl-no-unclassified = no-unclassified-canonical-question
 }

-- § Canonical closure: endpoint closed, remaining questions classified.
record CanonicalClosure : Setω where
 field
  cc-endpoint          : CanonicalClosedEndpoint
  cc-ledger            : CanonicalClosureLedger
  cc-set-like-burden   : SetLikeCompletionBurden
  cc-formulability-burden : FormulabilityBurden

open CanonicalClosure public

-- § The canonical closure of the checked route.
canonical-closure : CanonicalClosure
canonical-closure = record
 { cc-endpoint          = canonical-closed-endpoint
 ; cc-ledger            = canonical-closure-ledger
 ; cc-set-like-burden   = set-like-completion-burden
 ; cc-formulability-burden = formulability-burden
 }
```

26 Route-Admitted Completion Classification

The final closure theorem is not hidden in the prose. The paper now turns completion claims into a typed admission interface for this route. A route-admitted completion claim is one of four things: the canonical skeleton endpoint, an attempted full raw topos package, a set-like candidate above the endpoint, or a semantic-adequacy claim for the reading interface. Exhaustiveness is used here in that precise sense: every claim admitted by this route interface has one of the displayed classes. The wider foundation-admission classification is introduced in the next section.

The classification is then total by case analysis on that interface. The canonical skeleton is closed by the canonical-closure theorem. The raw topos case is blocked by the pullback obstruction. The set-like case is classified by its burden and by the extraction of binary distinction from object diversity. The semantic-adequacy case remains a named burden rather than an unclassified remainder.

```
-- § Finite admission interface for completion claims handled by this route.
data RouteCompletionExtension : Setω where
 route-canonical-skeleton : RouteCompletionExtension
 route-raw-topos-attempt  :
  ToposCompletionRequirements → RouteCompletionExtension
 route-set-like-candidate :
  SetLikeCompletionCandidate → RouteCompletionExtension
 route-semantic-adequacy :
  StableReadingSemanticAdequacy → RouteCompletionExtension

-- § Classification classes for the route-admitted completion interface.
```

```
data RouteCompletionClass : Set where
 route-canonical-skeleton-class : RouteCompletionClass
 route-raw-topos-blocked-class : RouteCompletionClass
 route-set-like-burden-class      : RouteCompletionClass
 route-semantic-adequacy-burden-class : RouteCompletionClass

-- § Evidence assigned to each classified route-admitted completion claim.
data ClassifiedRouteCompletion :
 RouteCompletionExtension → RouteCompletionClass → Setω where
 route-canonical-skeleton-classified :
  CanonicalClosure →
  ClassifiedRouteCompletion
   route-canonical-skeleton
   route-canonical-skeleton-class
 route-raw-topos-blocked-classified :
  (requirements : ToposCompletionRequirements) →
  ((requirements : ToposCompletionRequirements) → ⊥) →
  ClassifiedRouteCompletion
   (route-raw-topos-attempt requirements)
   route-raw-topos-blocked-class
 route-set-like-candidate-classified :
  (candidate : SetLikeCompletionCandidate) →
  EmbeddedBinaryDistinction
   (formulation-positions
    (category-formulation-interface
     (slc-category candidate)
     (slc-object-diversity candidate))) →
  ClassifiedRouteCompletion
   (route-set-like-candidate candidate)
   route-set-like-burden-class
 route-semantic-adequacy-classified :
  (adequacy : StableReadingSemanticAdequacy) →
  ClassifiedRouteCompletion
   (route-semantic-adequacy adequacy)
   route-semantic-adequacy-burden-class

-- § Classifier for the route-admitted completion interface.
route-completion-class : RouteCompletionExtension → RouteCompletionClass
route-completion-class route-canonical-skeleton =
 route-canonical-skeleton-class
route-completion-class (route-raw-topos-attempt requirements) =
 route-raw-topos-blocked-class
route-completion-class (route-set-like-candidate candidate) =
 route-set-like-burden-class
route-completion-class (route-semantic-adequacy adequacy) =
 route-semantic-adequacy-burden-class

-- § The raw full-topos case cannot be inhabited on this route.
route-raw-topos-attempt-impossible :
 (requirements : ToposCompletionRequirements) → ⊥
route-raw-topos-attempt-impossible =
 topos-completion-requirements-impossible

-- § Exhaustiveness: every route-admitted completion claim is classified.
route-completion-classified :
 (extension : RouteCompletionExtension) →
 ClassifiedRouteCompletion extension (route-completion-class extension)
route-completion-classified route-canonical-skeleton =
 route-canonical-skeleton-classified canonical-closure
route-completion-classified (route-raw-topos-attempt requirements) =
 route-raw-topos-blocked-classified
  requirements
  route-raw-topos-attempt-impossible
route-completion-classified (route-set-like-candidate candidate) =
 route-set-like-candidate-classified
  candidate
  (set-like-candidate-presupposes-distinction candidate)
route-completion-classified (route-semantic-adequacy adequacy) =
 route-semantic-adequacy-classified adequacy

-- § Package of the exhaustive route-admitted completion classification.
```

```
record ExhaustiveRouteCompletionClassification : Setω where
 field
  ercc-class : RouteCompletionExtension → RouteCompletionClass
  ercc-classified :
   (extension : RouteCompletionExtension) →
   ClassifiedRouteCompletion extension (ercc-class extension)
  ercc-raw-topos-attempt-impossible :
   (requirements : ToposCompletionRequirements) → ⊥
  ercc-set-like-to-distinction :
   (candidate : SetLikeCompletionCandidate) →
   EmbeddedBinaryDistinction
    (formulation-positions
     (category-formulation-interface
      (slc-category candidate)
      (slc-object-diversity candidate)))
  ercc-canonical-closure : CanonicalClosure

open ExhaustiveRouteCompletionClassification public

-- § The final classification theorem for completions admitted by this route.
route-completion-classification : ExhaustiveRouteCompletionClassification
route-completion-classification = record
 { ercc-class                       = route-completion-class
 ; ercc-classified                  = route-completion-classified
 ; ercc-raw-topos-attempt-impossible = route-raw-topos-attempt-impossible
 ; ercc-set-like-to-distinction     = set-like-candidate-presupposes-distinction
 ; ercc-canonical-closure           = canonical-closure
 }
```

27 Admitted Foundation Classification

The stronger sentence becomes exact by making admission exact. In this paper, a foundation claim is admitted when it either supplies a nontrivial interface of formulation or appears as one of the completion claims handled by the route above. The word "foundation" is therefore used through a formal interface: if a foundation enters as a nontrivial formulable foundation, it has already supplied two distinguishable positions; if it enters as a completion of the route, it is one of the route-admitted completion claims already classified.

With that admission boundary explicit, the classification theorem is total. Every admitted nontrivial formulable foundation is classified by the embedded binary distinction extracted from its own formulation positions. Every admitted completion claim is classified by the route-completion theorem. The maximally strong claim carried by this paper is therefore the formal classification of every foundation claim admitted as nontrivial and formulable, together with every completion claim admitted by this route.

```
-- § A nontrivial formulable foundation supplies formulation positions.
record FormulableFoundationCandidate : Set₁ where
 field
  ffc-interface : NontrivialFormulationInterface

open FormulableFoundationCandidate public

-- § Admitted foundation claims: entry-level foundations or route completions.
data AdmittedFoundationClaim : Setω where
 admitted-formulable-foundation :
  FormulableFoundationCandidate → AdmittedFoundationClaim
 admitted-route-completion :
  RouteCompletionExtension → AdmittedFoundationClaim

-- § Classification classes for admitted foundation claims.
data AdmittedFoundationClass : Set where
 foundation-distinctional-entry-class : AdmittedFoundationClass
 foundation-canonical-skeleton-class : AdmittedFoundationClass
```

```
 foundation-raw-topos-blocked-class : AdmittedFoundationClass
 foundation-set-like-burden-class : AdmittedFoundationClass
 foundation-semantic-adequacy-burden-class : AdmittedFoundationClass

-- § Route-completion classes become foundation classes at the completion level.
foundation-route-class : RouteCompletionClass → AdmittedFoundationClass
foundation-route-class route-canonical-skeleton-class =
 foundation-canonical-skeleton-class
foundation-route-class route-raw-topos-blocked-class =
 foundation-raw-topos-blocked-class
foundation-route-class route-set-like-burden-class =
 foundation-set-like-burden-class
foundation-route-class route-semantic-adequacy-burden-class =
 foundation-semantic-adequacy-burden-class

-- § Evidence that an admitted foundation claim has its assigned class.
data ClassifiedAdmittedFoundation :
 AdmittedFoundationClaim → AdmittedFoundationClass → Setω where
 admitted-formulable-foundation-classified :
  (foundation : FormulableFoundationCandidate) →
  EmbeddedBinaryDistinction
    (formulation-positions (ffc-interface foundation)) →
  ClassifiedAdmittedFoundation
    (admitted-formulable-foundation foundation)
    foundation-distinctional-entry-class
 admitted-route-completion-classified :
  (extension : RouteCompletionExtension) →
  ClassifiedRouteCompletion extension (route-completion-class extension) →
  ClassifiedAdmittedFoundation
    (admitted-route-completion extension)
    (foundation-route-class (route-completion-class extension))

-- § Classifier for admitted foundation claims.
admitted-foundation-class :
 AdmittedFoundationClaim → AdmittedFoundationClass
admitted-foundation-class (admitted-formulable-foundation foundation) =
 foundation-distinctional-entry-class
admitted-foundation-class (admitted-route-completion extension) =
 foundation-route-class (route-completion-class extension)

-- § Every admitted nontrivial formulable foundation carries distinction.
formulable-foundation-presupposes-distinction :
 (foundation : FormulableFoundationCandidate) →
 EmbeddedBinaryDistinction
  (formulation-positions (ffc-interface foundation))
formulable-foundation-presupposes-distinction foundation =
 formulation-interface-presupposes-distinction (ffc-interface foundation)

-- § Exhaustiveness: every admitted foundation claim is classified.
admitted-foundation-classified :
 (claim : AdmittedFoundationClaim) →
 ClassifiedAdmittedFoundation claim (admitted-foundation-class claim)
admitted-foundation-classified
 (admitted-formulable-foundation foundation) =
 admitted-formulable-foundation-classified
  foundation
  (formulable-foundation-presupposes-distinction foundation)
admitted-foundation-classified (admitted-route-completion extension) =
 admitted-route-completion-classified
  extension
  (route-completion-classified extension)

-- § Package of the exhaustive admitted-foundation classification.
record ExhaustiveAdmittedFoundationClassification : Setω where
 field
  eafc-class : AdmittedFoundationClaim → AdmittedFoundationClass
  eafc-classified :
    (claim : AdmittedFoundationClaim) →
    ClassifiedAdmittedFoundation claim (eafc-class claim)
  eafc-formulable-foundation-to-distinction :
    (foundation : FormulableFoundationCandidate) →
```

```
    EmbeddedBinaryDistinction
    (formulation-positions (ffc-interface foundation))
  eafc-route-completion-classification :
    ExhaustiveRouteCompletionClassification

open ExhaustiveAdmittedFoundationClassification public

-- § The admitted-foundation classification theorem.
admitted-foundation-classification :
 ExhaustiveAdmittedFoundationClassification
admitted-foundation-classification = record
 { eafc-class = admitted-foundation-class
 ; eafc-classified = admitted-foundation-classified
 ; eafc-formulable-foundation-to-distinction =
    formulable-foundation-presupposes-distinction
 ; eafc-route-completion-classification = route-completion-classification
 }
```

The burden is now located without treating the candidate package as a completed theorem. The terminal topos is not presented as the whole set-like universe. It is the first certified threshold; after that point, every completion claim admitted by this paper's route has a formal class. More generally, every admitted foundation claim has a formal class: either it is already distinctional at the level of formulability, or it is a completion claim classified by the route. A set-like categorical foundation extending the route would need to provide further nonterminal structure and a separate adequacy argument within that classified position.

Part V. Final theorem and reading

28 Final Result

The final statement packages three levels of the route.

At the raw level, stable readings carry real categorical structure: identities, composition, a terminal reading, weak binary products, and a displayed classifier extension. The same raw level also carries the formal obstruction. Global pullbacks cannot be supplied for all marker-preserving cospans, so the full raw elementary-topos requirements are blocked.

At the completion level, the quotient/skeleton carrier is the first certified topos stage. Its category is terminal, and the elementary topos data are supplied by direct construction. The canonical closure record then shows that the completed skeleton has no residual internal choice: presentations, objects, morphisms, finite-limit witnesses, classifier, and exponentials all reduce to the unique canonical witness.

Above the minimal topos threshold, the set-like and foundational claims enter through explicit admission interfaces. The set-like burden record shows why the terminal skeleton is not a nonterminal universe of formulable sets. The free completion record constructs the canonical well-pointed elementary topos with NNO over the generated reading skeleton and proves its initiality, uniqueness up to equivalence, and model classification. The formulability-burden record proves that any interface with two distinguishable formulation positions carries an embedded binary distinction. The route-completion and admitted-foundation classifications assign every admitted claim to its formal class.

This is the final theorem's force: in this construction, topos structure is generated after distinction and certified at completion; set-like and foundation-level extensions are then classified by the burdens through which they enter the route.

```
-- § Maximal raw structure established before quotient completion.
record RawStableReadingMaximalStructure : Setω where
```

```
  field
    rsm-category-data : (classifier : ClassifierObject) → CategoryData classifier
    rsm-terminal-reading : (classifier : ClassifierObject) →
                           StableReadingTerminal classifier
    rsm-weak-products : StableReadingWeakBinaryProducts
    rsm-product-uniqueness-boundary : Set₁
    rsm-subobject-candidate : (classifier : ClassifierObject) →
                              SubobjectClassifierCandidate classifier
    rsm-subobject-universal : StableReadingSubobjectClassifierUniversality
    rsm-no-global-pullbacks : StableReadingGlobalPullbacks → ⊥
    rsm-no-raw-topos-requirements : ToposCompletionRequirements → ⊥

open RawStableReadingMaximalStructure public

-- § The raw stage has structure, but its pullback obstruction is built in.
raw-stable-reading-maximal-structure : RawStableReadingMaximalStructure
raw-stable-reading-maximal-structure = record
  { rsm-category-data              = stable-reading-category-data
  ; rsm-terminal-reading           = stable-reading-terminal
  ; rsm-weak-products              = stable-reading-weak-products
  ; rsm-product-uniqueness-boundary = StableReadingProductUniquenessBoundary
  ; rsm-subobject-candidate        = classifier-subobject-candidate
  ; rsm-subobject-universal        = stable-reading-subobject-classifier-universality
  ; rsm-no-global-pullbacks        = raw-global-pullbacks-impossible
  ; rsm-no-raw-topos-requirements = topos-completion-requirements-impossible
  }

-- § Final theorem: raw structure, raw obstruction, completed topos, burden.
record VoidToposFinalTheorem : Setω where
  field
    vtf-boundary-map       : ToposBoundaryMap
    vtf-raw-structure      : RawStableReadingMaximalStructure
    vtf-raw-topos-blocked  : ToposCompletionRequirements → ⊥
    vtf-scope-conditions   : VoidToposScopeConditions
    vtf-completion-stage   : CompletionToposStage
    vtf-skeleton-completion : VoidSkeletonToposCompletion
    vtf-completed-topos    : ElementaryTopos (vst-category vtf-skeleton-completion)
    vtf-set-like-burden    : SetLikeCompletionBurden
    vtf-formulability-burden : FormulabilityBurden
    vtf-free-well-pointed-topos-with-nno : WellPointedToposWithNNOCompletion
    vtf-free-completion-initial :
      InitialCompletion vtf-free-well-pointed-topos-with-nno
    vtf-free-completion-unique-up-to-equivalence :
      (other : WellPointedToposWithNNOCompletion) →
      InitialCompletion other →
      CompletionEquivalence vtf-free-well-pointed-topos-with-nno other
    vtf-classifying-topos-of-reading-skeleton-models :
      ClassifyingToposOfReadingSkeletonModels
    vtf-canonical-closure  : CanonicalClosure
    vtf-route-completion-classification :
      ExhaustiveRouteCompletionClassification
    vtf-admitted-foundation-classification :
      ExhaustiveAdmittedFoundationClassification

open VoidToposFinalTheorem public

-- § The final witness packages the checked theorem and burden records.
void-topos-final-theorem : VoidToposFinalTheorem
void-topos-final-theorem = record
  { vtf-boundary-map       = topos-boundary-map
  ; vtf-raw-structure      = raw-stable-reading-maximal-structure
  ; vtf-raw-topos-blocked  = topos-completion-requirements-impossible
  ; vtf-scope-conditions   = void-topos-scope-conditions
  ; vtf-completion-stage   = completion-topos-stage
  ; vtf-skeleton-completion = void-skeleton-topos-completion
  ; vtf-completed-topos    = skeleton-elementary-topos
  ; vtf-set-like-burden    = set-like-completion-burden
  ; vtf-formulability-burden = formulability-burden
  ; vtf-free-well-pointed-topos-with-nno = free-well-pointed-topos-with-nno
  ; vtf-free-completion-initial = free-completion-initial
  ; vtf-free-completion-unique-up-to-equivalence =
```

```
    free-completion-unique-up-to-equivalence
  ; vtf-classifying-topos-of-reading-skeleton-models =
    free-classifying-topos-of-reading-skeleton-models
  ; vtf-canonical-closure  = canonical-closure
  ; vtf-route-completion-classification = route-completion-classification
  ; vtf-admitted-foundation-classification = admitted-foundation-classification
  }
```

29 Foundational Reading

The route

The checked route begins before an ambient topos is assumed. Its primitive data is a binary distinction. Distinctions normalize to Two up to boundary-preserving isomorphism, and the swapped presentation shows why that isomorphism must not be confused with raw fieldwise equality. The skeleton quotient supplies the lawful completed carrier for identifying presentations.

Stable readings inherit the normal form. At the raw reading level, categorical structure is generated and checked, but the split-truth cospan blocks global pullbacks. The raw layer is therefore not the topos stage. The topos stage appears when the distinction and reading presentations have been quotient-completed to the skeleton carrier. At that endpoint there is one object and one morphism, so the elementary topos structure closes by reflexivity.

The set-like question belongs above this threshold. A nonterminal set-like completion must supply object diversity and the further structure recorded in the candidate interface. Once such positions are supplied, the formulability-burden theorem extracts an embedded binary distinction. Thus, inside this route, set-like categorical structure is not the first account of formulability; it is an extension that carries the distinctional data under which its own language can be used.

Reading the theorem

The theorem shifts the burden of justification. If one proposes a categorical universe as the first account of formulation, with objects, morphisms, equality, truth values, pullbacks, and classifiers already in place, this route asks where the distinguishable formulation positions enter. When they are supplied, the route extracts binary distinction; when they are not supplied, the proposal has not yet entered the paper's admission interface.

Categorical and set-like foundations are therefore repositioned rather than dismissed. They can appear as completed stages, and the free well-pointed completion shows one canonical syntactic way to inhabit the set-like burden. What they cannot do inside this route is erase the priority of the distinctional and quotient structures by which the route gets to the topos level.

The scope is part of the rigor of the result. The marker interface is a formal stability interface. The completion constructed here is minimal and terminal. Richer quotients and semantic adequacy remain possible as further constructions, but inside the route-completion and admitted-foundation interfaces of this paper, their positions are classified. The result is the checked classification of the doctrines that enter through this route's formal gates.